

What is React?

React is a front-end JavaScript library.

React was developed by the Facebook Software Engineer Jordan Walke.

React is also known as React.js or ReactJS.

React is a tool for building UI components.

How does React Work?

React creates a VIRTUAL DOM in memory.

Instead of manipulating the browser's DOM directly, React creates a virtual DOM in memory, where it does all the necessary manipulating, before making the changes in the browser DOM.

React only changes what needs to be changed!

React finds out what changes have been made, and changes **only** what needs to be changed.

Setting up a React Environment

First, make sure you have Node.js installed. You can check by running this in your terminal:

```
node -v
```

If Node.js is installed, you will get a result with the version number:

```
v22.15.0
```

If not, you will need to install Node.js.

Install a Build Tool (Vite)

When you have Node.js installed, you can start creating a React application by choosing a build tool.

We will use the Vite build tool in this tutorial.

Run this command to install Vite:

```
npm install -g create-vite
```

If the installation was a success, you will get a result like this:

```
added 1 package in 649ms
```

Create a React Application

Run this command to create a React application named my-react-app:

```
npm create vite@latest my-react-app -- --template react
```

If you get this message, just press y and press Enter to continue:

Need to install the following packages:

```
create-vite@9.0.7
```

```
Ok to proceed? (y)
```

Next, you might get this message, just press Enter to continue:

```
> npx
```

```
> create-vite my-react-app --template react
```

```
|
```

```
* Install with npm and start now?
```

```
| > Yes / No
```

```
-
```

If the creation was a success, you will get a result like this:

VITE v8.0.15 ready in 262 ms

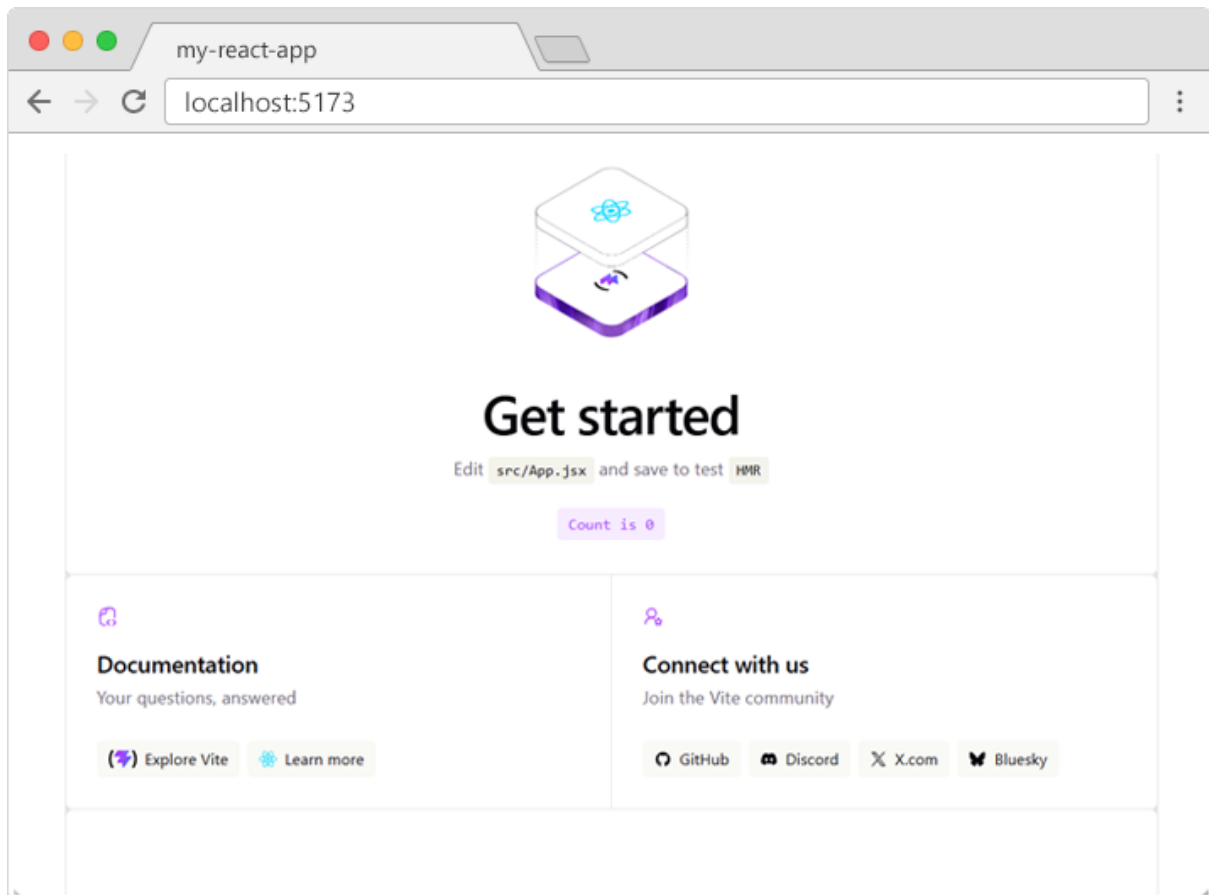
→ Local: <http://localhost:5173/>

→ Network: use `--host` to expose

→ press `h` + enter to show help

A new browser window will pop up with your newly created React App! If not, open your browser and type `localhost:5173` in the address bar.

The result:



React Render HTML

React renders HTML to the web page via a container, and a function called `createRoot()`.

To better understand the content of the `index.html` file, let's remove all the code we don't need.

Example

The `index.html` file should now look like this:

`index.html`

```
<!doctype html>

<html lang="en">

  <body>

    <div id="root"></div>

    <script type="module" src="/src/main.jsx"></script>

  </body>

</html>
```

The file is now stripped from unnecessary code, and we can concentrate on learning React without any disturbing elements.

The createRoot Function

The createRoot function is located in the main.jsx file in the src folder, and is a built-in function that is used to create a root node for a React application.

Example

The default content of the src/main.jsx file:

main.jsx

```
import { StrictMode } from 'react'

import { createRoot } from 'react-dom/client'

import './index.css'

import App from './App.jsx'

createRoot(document.getElementById('root')).render(
```

```
<StrictMode>

  <App />

</StrictMode>

)
```

The `createRoot()` function takes one argument, an HTML element.

The purpose of the function is to define the HTML element where a React component should be displayed.

`react-dom/client`

This is a module provided by React.

`react-dom/client`

contains functions that allow React to **connect your React application to the HTML page in the browser**.

For example, your HTML might contain:

```
<div id="root"></div>
```

React needs to know where it should display your components. That's where `createRoot()` comes in.

2. `createRoot`

`createRoot` is a function provided by `react-dom/client`.

We use it like this:

```
const root = createRoot(document.getElementById('root'));
```

This means:

"Create a React root inside the HTML element whose ID is root."

Then we can tell React what to display:

```
root.render(<App />);
```

So:

```
const root = createRoot(document.getElementById('root'));
```

```
root.render(<App />);
```

means:

Find `<div id="root">`, create a React application there, and render the App component inside it.

```
{ } around createRoot
```

This:

```
import { createRoot } from 'react-dom/client'
```

is a **named import**.

A **named import** means you import a specific thing by its **exact exported name** from another JavaScript file/module.

Simple example

Suppose `math.js` has:

```
export const add = (a, b) => a + b;
```

```
export const subtract = (a, b) => a - b;
```

Now in another file:

```
import { add } from './math.js';
```

This is a **named import**.

We are saying:

"From `math.js`, give me the thing named `add`."

You can import multiple named exports:

```
import { add, subtract } from './math.js';
```

Why { }?

The curly braces indicate a named import:

```
import { add } from './math.js';
```

↑

named import

Compare that with a **default import**:

```
import add from './math.js';
```

No `{ }` is used for a default import.

Easy way to remember

Named import:

```
import { name } from "module";
```

↑

exact name

Default import:

```
import name from "module";
```

↑

can choose name

So in our React code:

```
import { createRoot } from 'react-dom/client'
```

createRoot is a **named export**, and you're importing it by its exact name.

It means:

"From the react-dom/client module, give me the thing named createRoot."

The complete React example

```
import { createRoot } from 'react-dom/client';
```

```
function App() {  
  return <h1>Hello React!</h1>;  
}
```

```
const root = createRoot(document.getElementById('root'));
```

```
root.render(<App />);
```

And your HTML:

```
<div id="root"></div>
```

The result in the browser is:

Hello React!

In one sentence

```
import { createRoot } from 'react-dom/client'
```

means:

"Import React's createRoot function so I can create a React root in the browser and render my React components into it."

To better understand the `createRoot` function, let's remove unnecessary code and write our own "Hello React!" example:

Example

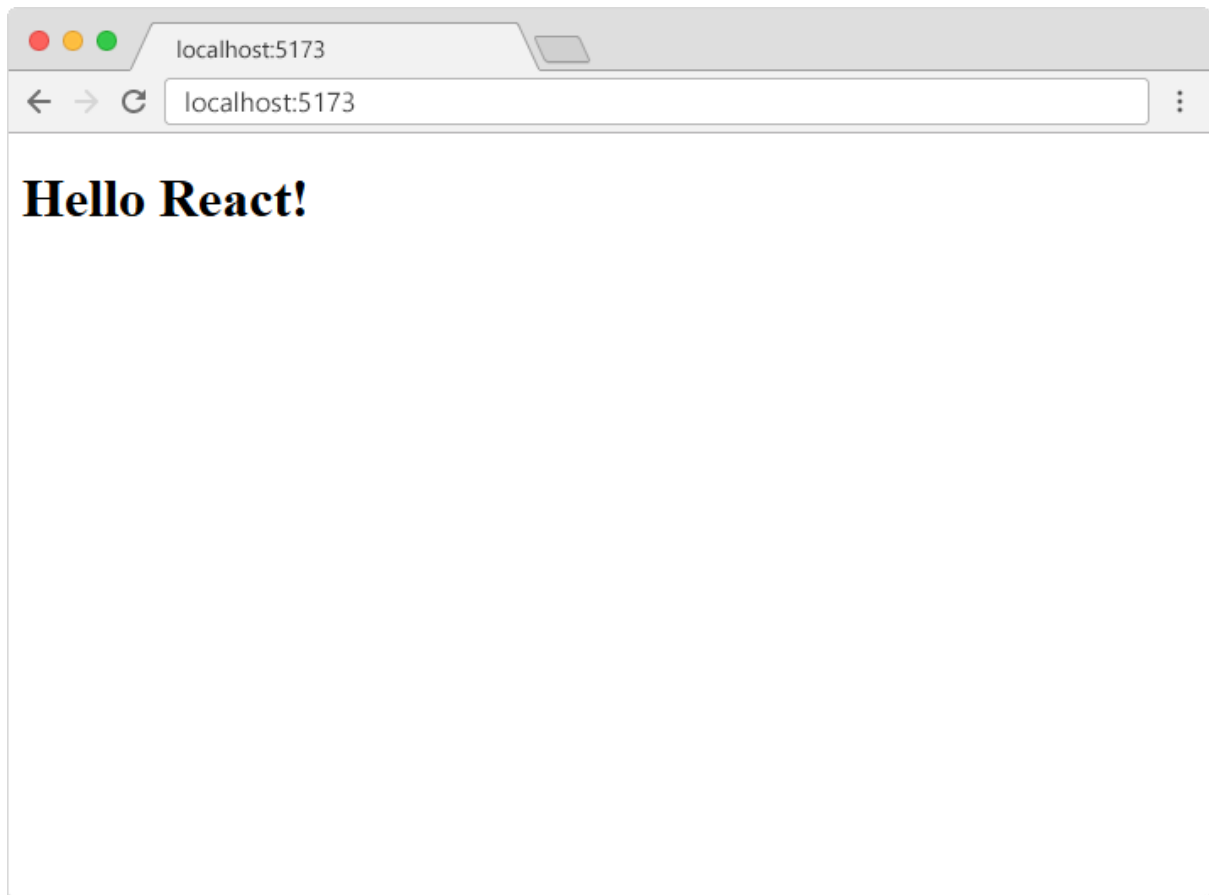
The `src/main.jsx` file should now look like this:

`main.jsx`

```
import { createRoot } from 'react-dom/client'

createRoot(document.getElementById('root')).render(
  <h1>Hello React!</h1>
)
```

If you save the file, the result in the browser will look like this:



The render Method

The render method defines what to render in the HTML container.

The result is displayed in the `<div id="root">` element.

Example

Display a paragraph inside the "root" element:

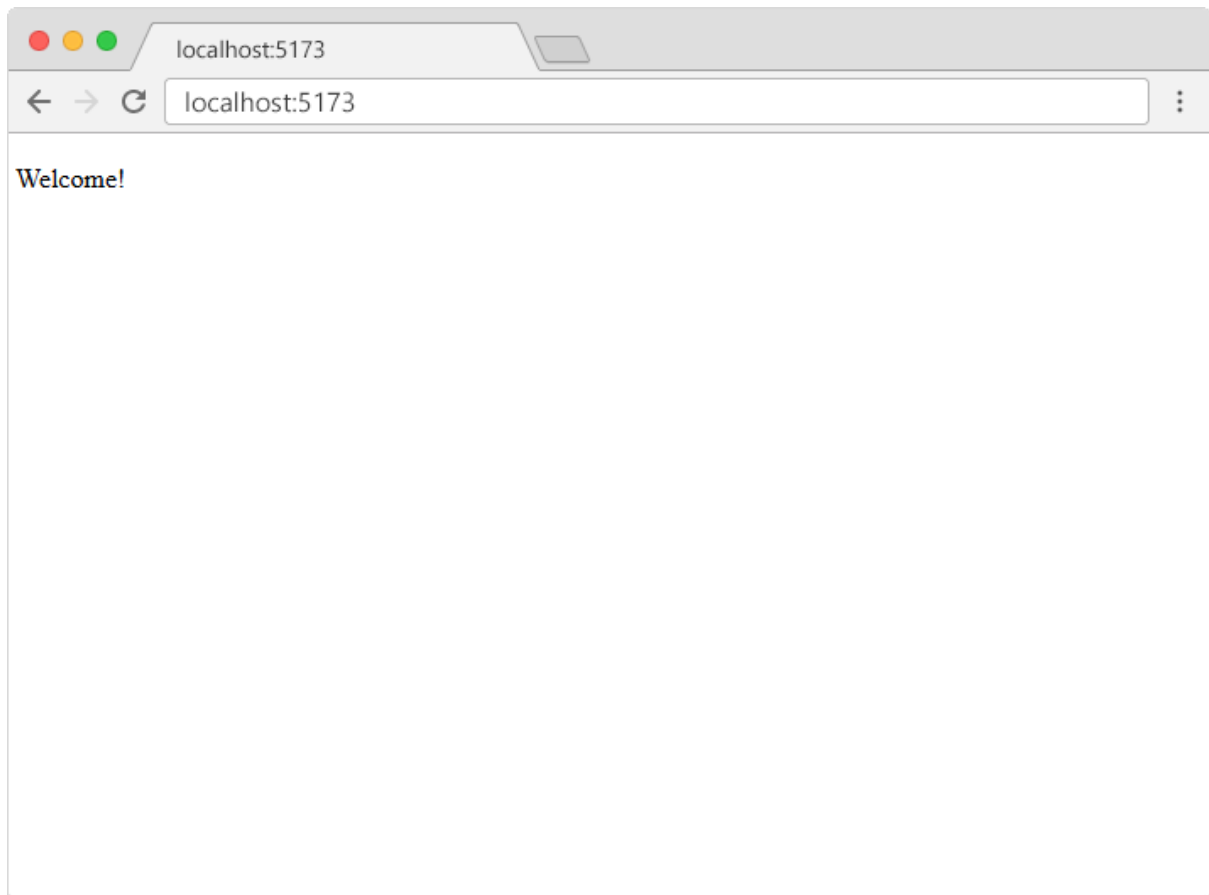
main.jsx

```
import { createRoot } from 'react-dom/client'
```

```
createRoot(document.getElementById('root')).render(  
  <p>Welcome!</p>  
)
```

)

The result will look like this:



The Container

React uses a container to render HTML in a web page.

Typically, this container is a `<div id="root"></div>` element in the `index.html` file.

React ES6

What is ES6?

ES6 stands for ECMAScript 6.

ECMAScript was created to standardize JavaScript, and ES6 is the 6th version of ECMAScript, it was published in 2015, and is also known as ECMAScript 2015.

Why Should I Learn ES6?

React uses ES6, and you should be familiar with some of the new features like:

- Classes
- Arrow Functions
- Variables (let, const, var)
- Array Methods like .map()
- Destructuring
- Modules
- Ternary Operator
- Spread Operator

ES6 introduced classes.

A class is a type of function, but instead of using the keyword function to initiate it, we use the keyword class, and the properties are assigned inside a constructor() method.

A simple class constructor:

```
class Car {  
  constructor(name) {  
    this.brand = name;  
  }  
}
```

Notice the case of the class name. We have begun the name, "Car", with an uppercase character. This is a standard naming convention for classes.

Now you can create objects using the Car class:

Example

Create an object called "mycar" based on the Car class:

```
class Car {  
  constructor(name) {  
    this.brand = name;  
  }  
}  
  
const mycar = new Car("Ford");
```

Note: The constructor function is called automatically when the object is initialized.

Method in Classes

You can add your own methods in a class:

Example

Create a method named "present":

```
class Car {  
  constructor(name) {  
    this.brand = name;  
  }  
  
  present() {  
    return 'I have a ' + this.brand;  
  }  
}
```

```
const mycar = new Car("Ford");  
  
mycar.present();
```

As you can see in the example above, you call the method by referring to the object's method name followed by parentheses (parameters would go inside the parentheses).

Class Inheritance

To create a class inheritance, use the extends keyword.

A class created with a class inheritance inherits all the methods from another class:

Example

Create a class named "Model" which will inherit the methods from the "Car" class:

```
class Car {  
  
  constructor(name) {  
    this.brand = name;  
  }  
  
  present() {  
    return 'I have a ' + this.brand;  
  }  
}  
  
class Model extends Car {  
  constructor(name, mod) {  
    super(name);  
    this.model = mod;  
  }  
}
```

```
show() {  
    return this.present() + ', it is a ' + this.model  
}  
}  
  
const mycar = new Model("Ford", "Mustang");  
mycar.show();
```

The `super()` method refers to the parent class.

By calling the `super()` method in the constructor method, we call the parent's constructor method and get access to the parent's properties and methods.

Arrow Functions

Arrow functions allow us to write shorter function syntax:

```
hello = function() {  
    return "Hello World!";
```

```
}
```

Example

With Arrow Function:

```
hello = () => {  
    return "Hello World!";
```

```
}
```

It gets shorter! If the function has only one statement, and the statement returns a value, you can remove the brackets *and* the return keyword:

Example

Arrow Functions Return Value by Default:

```
hello = () => "Hello World!";
```

Note: This works only if the function has only one statement.

If you have parameters, you pass them inside the parentheses:

Example

Arrow Function With Parameters:

```
hello = (val) => "Hello " + val;
```

In fact, if you have only one parameter, you can skip the parentheses as well:

Example

Arrow Function Without Parentheses:

```
hello = val => "Hello " + val;
```

What About this?

The handling of this is also different in arrow functions compared to regular functions.

In short, with arrow functions there is no binding of this.

In regular functions the `this` keyword represented the object that called the function, which could be the window, the document, a button or whatever.

With arrow functions, the `this` keyword *always* represents the object that defined the arrow function.

Let us take a look at two examples to understand the difference.

Both examples call a method twice, first when the page loads, and once again when the user clicks a button.

The first example uses a regular function, and the second example uses an arrow function.

The result shows that the first example returns two different objects (window and button), and the second example returns the Header object twice.

Example

With a regular function, this represents the object that called the function:

```
class Header {  
  constructor() {  
    this.color = "Red";  
  }  
}
```

//Regular function:

```
changeColor = function() {  
  document.getElementById("demo").innerHTML += this;  
}  
}
```

```
const myheader = new Header();
```

//The window object calls the function:

```
window.addEventListener("load", myheader.changeColor);
```

//A button object calls the function:

```
document.getElementById("btn").addEventListener("click", myheader.changeColor);
```

Regular Function

The **this** keyword represents different objects depending on how the function was called.

Click Me!

this represents:

[object Window]

See the difference before and after the button is clicked.

Regular Function

The **this** keyword represents different objects depending on how the function was called.

Click Me!

this represents:

[object Window][object HTMLButtonElement]

See the difference before and after the button is clicked.

Example

With an arrow function, this represents the Header object no matter who called the function:

```
class Header {  
  constructor() {  
    this.color = "Red";  
  }  
}
```

//Arrow function:

```
changeColor = () => {  
  document.getElementById("demo").innerHTML += this;  
}
```

```
}  
}
```

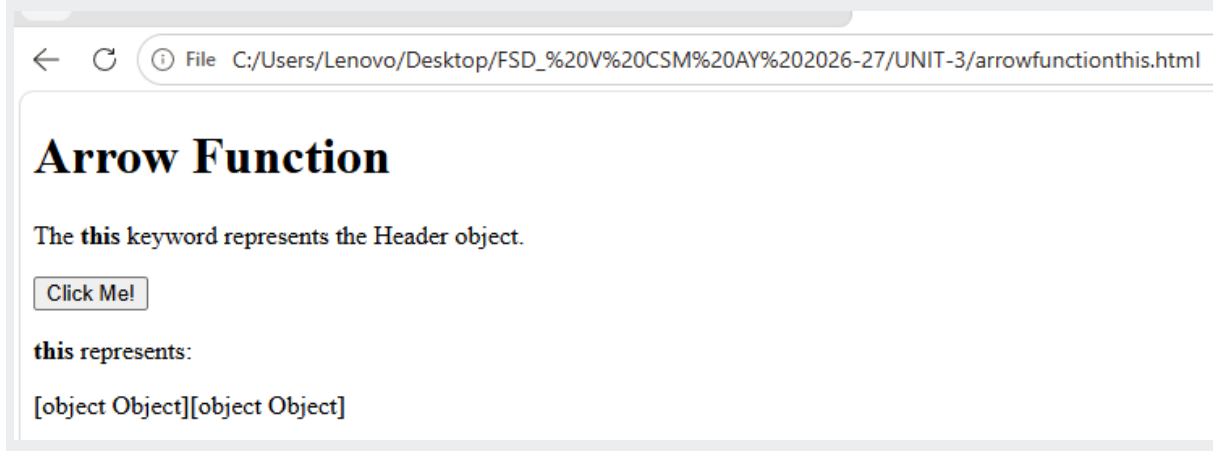
```
const myheader = new Header();
```

```
//The window object calls the function:
```

```
window.addEventListener("load", myheader.changeColor);
```

```
//A button object calls the function:
```

```
document.getElementById("btn").addEventListener("click", myheader.changeColor);
```



output:

```
[object object]
```

```
document.getElementById("demo").innerHTML += this;
```

At this point, this refers to your Header object:

```
const myheader = new Header();
```

So JavaScript is effectively doing:

```
document.getElementById("demo").innerHTML += myheader;
```

But innerHTML expects **text**, not an object.

When JavaScript needs to convert an ordinary object to text, it uses the object's default toString() method:

```
myheader.toString()
```

which produces:

```
[object Object]
```

our object is:

```
{  
  color: "Red"  
}
```

JavaScript converts it:

Object → String

and the default conversion gives:

```
[object Object]
```

So:

this

becomes:

```
[object Object]
```

Why doesn't it show Red?

Because the object contains:

```
this.color = "Red";
```

but you're printing the **whole object**, not its color property.

If you change:

```
innerHTML += this;
```

to:

```
innerHTML += this.color;
```

the output will be:

```
Red
```

And if you want to see the object itself more clearly, use:

```
console.log(this);
```

Then the browser console will show something similar to:

```
Header { color: "Red", changeColor: ... }
```

So

this → **object** → **converted to string** → [object Object]

this.color → "Red"

JavaScript Array map()

The map() Method

The map() method creates a new array with the results of calling a function for every array element.

Example

Multiply each number by 2:

```
<!DOCTYPE html>
<html>
<body>

<div id="demo"></div>

<script>
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(x => x * 2);

document.getElementById("demo").innerHTML = doubled;
</script>

</body>
</html>
```

Output:2,4,6,8

map() in React

The map() method is commonly used in React to render lists of elements:

index.html

```
<!doctype html>
<html lang="en">
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Main.jsx

```
import { createRoot } from 'react-dom/client'

const fruitlist = ['apple', 'banana', 'cherry'];

function MyList() {
  return (
    <ul>
      {fruitlist.map(fruit =>
        <li key={fruit}>{fruit}</li>
      )}
    </ul>
  );
}

createRoot(document.getElementById('root')).render(
  <MyList />
```

- apple
- banana
- cherry

map() used with array of objects

index.html

```
<!doctype html>
<html lang="en">
  <body>
    <div id="root"></div>
```

```
<script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

```
import { createRoot } from 'react-dom/client'

const users = [
  { id: 1, name: 'John', age: 30 },
  { id: 2, name: 'Jane', age: 25 },
  { id: 3, name: 'Bob', age: 35 }
];

function UserList() {
  return (
    <ul>
      {users.map(user =>
        <li key={user.id}>
          {user.name} is {user.age} years old
        </li>
      )}
    </ul>
  );
}

createRoot(document.getElementById('root')).render(
  <UserList />
)
```

- John is 30 years old
- Jane is 25 years old
- Bob is 35 years old

Destructuring in React

Destructuring is a JavaScript feature that allows you to extract values from objects or arrays into distinct variables. In React, it's commonly used with props, hooks, and state management.

Destructuring Arrays

Here is the old way of assigning array items to a variable:

```
<!DOCTYPE html>

<html>
```

```
<body>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const vehicles = ['mustang', 'f-150', 'expedition'];
```

```
// old way
```

```
const car = vehicles[0];
```

```
const truck = vehicles[1];
```

```
const suv = vehicles[2];
```

```
//You can now access each variable separately:
```

```
document.getElementById('demo').innerHTML = truck;
```

```
</script>https://www.w3schools.com/
```

```
</body>
```

```
</html>
```

output: f-150

Here is the new way of assigning array items to a variable:

Example

With destructuring:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const vehicles = ['mustang', 'f-150', 'expedition'];
```

```
const [car, truck, suv] = vehicles;
```

```
//You can now access each variable separately:
```

```
document.getElementById('demo').innerHTML = truck;
```

```
</script>
```

```
</body>
```

```
</html>
```

output:f-150

When destructuring *arrays*, the order that variables are declared is important.

If we only want the car and suv we can simply leave out the truck but keep the comma:

```
const vehicles = ['mustang', 'f-150', 'expedition'];
```

```
const [car,, suv] = vehicles;
```

Destructuring comes in handy when a function returns an array:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<p id="demo"></p>
```

```
<script>
```

```
function dateInfo(dat) {
```

```
    const d = dat.getDate();
```

```
    const m = dat.getMonth() + 1;
```

```
    const y = dat.getFullYear();
```

```
    return [d, m, y];
```

```
}
```

```
const [date, month, year] = dateInfo(new Date());
```

```
//You can now access each variable separately:
```

```
document.getElementById('demo').innerHTML = "The year is " + year;
```

```
</script>
```

```
</body>
```

```
</html>
```

output:

The year is 2026

Destructuring in React Components

Destructuring is particularly useful in React for working with props, hooks, and API responses. It helps make your code more concise and easier to read.

Props Destructuring

When a component receives props, you can use destructuring to extract the values you need.

Check out the difference between using and not using destructuring:

```
//Using destructuring:
function Greeting({ name, age }) {
  return <h1>Hello, {name}! You are {age} years old.</h1>;
}

//NOT using destructuring:
function Greeting(props) {
  return <h1>Hello, {props.name}! You are {props.age} years old.</h1>;
}
```

Main.jsx

```
import { createRoot } from 'react-dom/client'

function Greeting({ name, age }) {
  return <h1>Hello, {name}! You are {age} years old.</h1>;
}

createRoot(document.getElementById('root')).render(
  <Greeting name="John" age={25} />
);
```

Index.html

```
<!doctype html>
<html lang="en">
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Output:

Hello, John! You are 25 years old.

useState Hook Destructuring

When a component uses the *useState* hook, we use destructuring to extract the values from it.

Example

Using destructuring to extract values from useState:

Main.jsx

```
import { createRoot, useState } from 'react-dom/client'

function Counter() {
  // Destructuring the array returned by useState
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}

createRoot(document.getElementById('root')).render(
  <Counter />
);
```

Count: 1

```
<!doctype html>
<html lang="en">
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Spread Operator

The JavaScript spread operator (...) copies all or part of an existing array or object into another array or object.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<script>
```

```
const numbersOne = [1, 2, 3];
```

```
const numbersTwo = [4, 5, 6];
```

```
const numbersCombined = [...numbersOne, ...numbersTwo];
```

```
document.write(numbersCombined);
```

```
</script>
```

```
</body>
```

```
</html>
```

output:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<script>
```

```
const numbersOne = [1, 2, 3];
```

```
const numbersTwo = [4, 5, 6];  
const numbersCombined = [...numbersOne, ...numbersTwo];
```

```
document.write(numbersCombined);  
</script>
```

```
</body>  
</html>
```

output:
1,2,3,4,5,6

The spread operator is often used in combination with destructuring.

```
<!DOCTYPE html>  
<html>
```

```
<body>
```

```
<script>
```

```
const numbers = [1, 2, 3, 4, 5, 6];
```

```
const [one, two, ...rest] = numbers;
```

```
document.write("<p>" + one + "</p>");  
document.write("<p>" + two + "</p>");  
document.write("<p>" + rest + "</p>");  
</script>
```

```
</body>
```

```
</html>
```

output:

1

2

3,4,5,6

We can use the spread operator with objects too:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<script>
```

```
const myVehicle = {
```

```
  brand: 'Ford',
```

```
  model: 'Mustang',
```

```
  color: 'red'
```

```
}
```

```
const updateMyVehicle = {
```

```
  type: 'car',
```

```
  year: 2021,
```

```
  color: 'yellow'
```

```
}
```

```
const myUpdatedVehicle = {...myVehicle, ...updateMyVehicle}
```

//Check the result object in the console:

```
console.log(myUpdatedVehicle);
```

```
</script>
```

<p>Press F12 and see the result object in the console view.</p>

```
</body>
```

```
</html>
```

Modules

JavaScript modules allow you to break up your code into separate files.

This makes it easier to maintain the code-base.

ES Modules rely on the import and export statements.

Ternary Operator

The ternary operator is a simplified conditional operator like if / else.

Syntax: condition ? <expression if true> : <expression if false>

Here is an example using if / else:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1 id="demo"></h1>
```

```
<script>
```

```
function renderApp() {  
  document.getElementById("demo").innerHTML = "Welcome!";  
}  
  
function renderLogin() {  
  document.getElementById("demo").innerHTML = "Please log in";  
}  
  
let authenticated = true;  
  
authenticated ? renderApp() : renderLogin();
```

```
</script>
```

<p>Try changing the "authenticated" variable to false, and run the code to see what happens.</p>

```
</body>
```

```
</html>
```

Try changing the "authenticated" variable to false, and run the code to see what happens

Template Strings

Template strings allow you to write strings that span multiple lines and include embedded expressions:

Example

Before:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>Example</h1>
```

```
<p>Without using template strings:</p>
```

```
<div id="demo"></div>
```

```
<script>
```

```
const name = "John";
```

```
const age = 30;
```

```
const message = "Hello, " + name + "!\n" +
```

```
"You are " + age + " years old.";
```

```
document.getElementById("demo").innerHTML = message;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example

Without using template strings:

Hello, John! You are 30 years old.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>Example</h1>
```

```
<p>With template strings you can write strings that span multiple lines and include  
embedded expressions:</p>
```

```
<div id="demo"></div>
```

```
<script>
```

```
const name = "John";
```

```
const age = 30;
```

```
const message = `Hello, ${name}!
```

```
You are ${age} years old.`;
```

```
document.getElementById("demo").innerHTML = message;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example

With template strings you can write strings that span multiple lines and include embedded expressions:

Hello, John! You are 30 years old.

Template strings use backticks (`) instead of quotes and can include:

- Multiple lines without \n
- Expressions inside \${}
- Quotes without escaping

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>Example</h1>
```

```
<p>With template strings you can write strings that span multiple lines:</p>
```

```
<div id="demo"></div>
```

```
<script>
```

```
const html = `
```

```
<div>
```

```
<h1>Title</h1>
```

```
<p>Paragraph</p>
```

```
</div>
```

```
`;
```

```
document.getElementById("demo").innerHTML = html;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example

With template strings you can write strings that span multiple lines:

Title

Paragraph

The indentation in multi-line strings becomes part of the string.

Example

The indentation becomes part of the string:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>Example</h1>
```

```
<p>With template strings, the indentation is part of the string.</p>
```

```
<p>This is not visible for most HTML elements, but the PRE element preserves any spaces  
and new lines:</p>
```

```
<pre id="demo"></pre>
```

```
<script>
```

```
const x = `
```

```
  John:
```

Hello, how are you?

Jane:

I'm fine, thanks!

`;
`;

```
document.getElementById("demo").innerHTML = x;
```

```
</script>
```

```
</body>
```

```
</html>
```

Example

With template strings, the indentation is part of the string.

This is not visible for most HTML elements, but the PRE element preserves any spaces and new lines:

```
John:
  Hello, how are you?
Jane:
  I'm fine, thanks!
```

React JSX

What is JSX?

JSX stands for JavaScript XML.

JSX allows us to write HTML in React.

JSX makes it easier to write and add HTML in React.

Coding JSX

JSX allows us to write HTML elements in JavaScript and place them in the DOM without any createElement() and/or appendChild() methods.

JSX converts HTML tags into react elements.

Example 1

JSX:

main.jsx

```
const myElement = <h1>I Love JSX!</h1>;
```

```
createRoot(document.getElementById('root')).render(  
  myElement  
);
```

```
<!doctype html>
```

```
<html lang="en">
```

```
<body>
```

```
<div id="root"></div>
```

```
<script type="module" src="/src/main.jsx"></script>
```

```
</body>
```

```
</html>
```

Example 2

Without JSX:

main.jsx

```
import React from 'react';
```

```
import { createRoot, createElement } from 'react-dom/client'
```

```
const myElement = React.createElement('h1', {}, 'I do not use JSX!');
```

```
createRoot(document.getElementById('root')).render(
```

```
  myElement
```

```
);
```

```
<!doctype html>
```

```
<html lang="en">
```

```
  <body>
```

```
    <div id="root"></div>
```

```
    <script type="module" src="/src/main.jsx"></script>
```

```
  </body>
```

```
</html>
```

React JSX Expressions

One of the most powerful features of JSX is the ability to embed JavaScript expressions directly within your markup.

Expressions

You can insert any valid JavaScript expression inside JSX by wrapping it in curly braces { }.

React will evaluate the expression and render the result in the DOM.

Execute the expression $218 * 1.36$:

```
function Car() {  
  return (  
    <>  
      <h1>My car</h1>  
      <p>It has {218 * 1.36} horsepower</p>  
    </>  
  );  
}
```

Variables

Variables are also valid expressions. Insert variables in JSX by wrapping it in curly braces { }.

Example

Use a variable inside JSX:

```
function Car() {  
  
  const hp = 218 * 1.36;  
  
  return (  
  
    <>  
  
    <h1>My car</h1>  
  
    <p>It has {hp} horsepower</p>  
  
    </>  
  
  );  
  
}
```

Function Calls

Function calls are valid expressions. Insert function calls in JSX by wrapping it in curly braces { }.

Example

Use a function inside JSX:

```
function kwtohp(kw) {  
  
  return kw * 1.36;  
  
}
```

```
function Car() {  
  
  return (  
  
    <>  
  
    <h1>My car</h1>  
  
    <p>It has {kwtohp(218)} horsepower</p>  
  
    </>  
  
  );  
  
}
```

Object Properties

Access object properties within JSX:

Example

Refer to an object property inside JSX:

```
function Car() {  
  
  const myobj = {  
  
    name: "Fiat",
```

```
    model: "500",

    color: "white"

  };

  return (

    <>

    <h1>My car is a {myobj.color} {myobj.name} {myobj.model}</h1>

    </>

  );
}
```

JavaScript Array map()

The map() Method

The map() method creates a new array with the results of calling a function for every array element.

Example [Get your own React.js Server](#)

Multiply each number by 2:

```
const numbers = [1, 2, 3, 4];

const doubled = numbers.map(x => x * 2);
```

map() in React

The map() method is commonly used in React to render lists of elements:

Example

```
const fruitlist = ['apple', 'banana', 'cherry'];

function MyList() {
  return (
    <ul>
      {fruitlist.map(fruit =>
        <li key={fruit}>{fruit}</li>
      )}
    </ul>
  );
}
```

React.js Installation and Execution in Visual Studio Code (Step-by-Step Guide)

Step 1: Install Node.js

React requires **Node.js**.

1. Open your browser.
2. Go to:

<https://nodejs.org>

3. Download the **LTS (Long Term Support)** version.
4. Install it using the default settings.
5. Restart your computer if prompted.

Step 2: Verify Node.js Installation

Open **Command Prompt** or **VS Code Terminal**.

Type:

```
node -v
```

Example output:

```
v22.18.0
```

Check npm:

```
npm -v
```

Example output:

```
10.9.3
```

If both commands display version numbers, Node.js is installed correctly.

Step 3: Open Visual Studio Code

Open **Visual Studio Code**.

Step 4: Open a Terminal

Go to

Terminal → New Terminal

A terminal opens at the bottom.

Step 5: Create a React Project

Type: IF NO ERROR TYPE: npm create vite@latest

IF ERROR TYPE

npm.cmd create vite@latest

Press **Enter**.

Step 6: Enter Project Name

It asks:

Project name:

Example:

my-react-app

Press **Enter**.

Step 7: Select Framework

You will see:

Select a framework

- Vanilla
- Vue
- React
- Preact
- Lit
- Svelte

Use the **Arrow Keys**.

Select

React

Press **Enter**.

Step 8: Select Variant

Next screen:

Select a variant

› JavaScript
TypeScript

Choose

JavaScript

Press **Enter**.

Step 9: Select Linter

Next screen:

Select a linter

› ESLint
Biome
None

Choose

ESLint

Press **Enter**.

Step 10: Move into the Project Folder

Suppose the project name is

my-react-app

Type

cd my-react-app

Step 11: Install Required Packages

Type

Npm.cmd install

Wait until installation completes.

Example:

added 152 packages

Step 12: Start the React Development Server

Type

Npm.cmd run dev

You will see:

VITE v7.x.x

Local:

<http://localhost:5173/>

Sometimes the port may be:

5174

5175

5176

Always open the **exact URL** shown in your terminal.

Step 13: Open the Browser

Open

<http://localhost:5173/>

(or whatever URL Vite displays).

You will see the default React page.

Step 14: Folder Structure

Your project should look like this:

```
my-react-app
├── node_modules
├── public
├── src
│   ├── App.jsx
│   ├── App.css
│   ├── main.jsx
│   └── assets
```



- package.json
- package-lock.json
- vite.config.js
- eslint.config.js

Step 15: Open App.jsx

In VS Code, open:

src
↓
App.jsx

Step 16: Replace the Code

Delete the existing code and paste:

```
function App() {  
  return (  
    <div>  
      <h1>Welcome to React.js</h1>  
      <h2>My First React Application</h2>  
      <p>Hello Students!</p>  
    </div>  
  );  
}  
  
export default App;
```

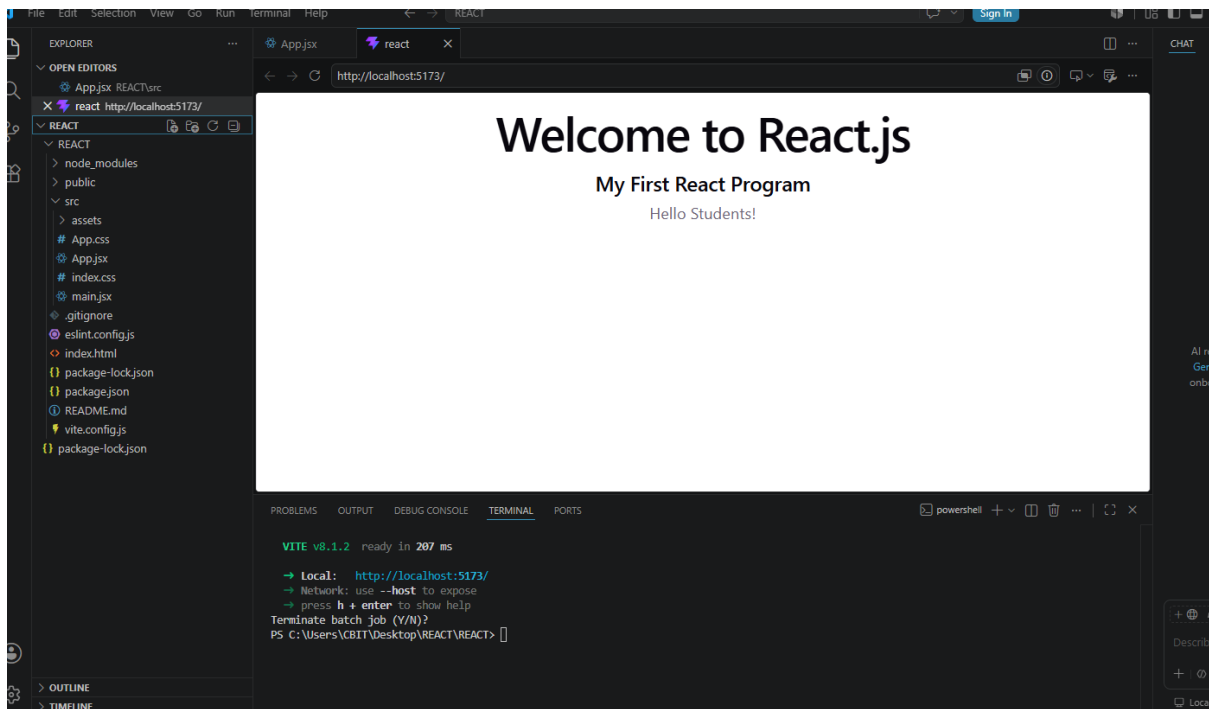
Step 17: Save the File

Press

Ctrl + S

The browser refreshes automatically.

Output



Welcome to React.js

My First React Application

Hello Students!

Step 18: Stop the Server

Press

Ctrl + C
in the terminal.

Running the Project Again

The next time you use the same project:

Open VS Code.

Go to the project folder:

```
cd my-react-app
```

Start the server:

```
npm run dev
```

Open the URL displayed in the terminal.

You do not need to run npm install again unless you delete the node_modules folder or create a new project.

Useful Commands

Command	Purpose
node -v	Check Node.js version
npm -v	Check npm version
npm create vite@latest	Create a new React project
cd my-react-app	Enter the project folder
npm install	Install project dependencies
npm run dev	Start the React development server
Ctrl + C	Stop the development server

Common Errors and Solutions

Error: npm is not recognized

Cause: Node.js is not installed or not added to the system PATH.

Solution: Install the latest LTS version of Node.js and restart your computer.

Error: ERR_CONNECTION_REFUSED

Cause: The React development server is not running.

Solution:

npm run dev

Then open the URL shown in the terminal.

Error: Port already in use

Cause: Another application is using the default port.

Solution: Vite automatically selects another port (for example, 5174 or 5175). Open the new URL displayed in the terminal.

Error: Cannot find module

Solution:

Npm.cmd install

PS C:\Users\CBIT\Desktop\getexpress> npm.cmd create vite@latest my-react-app

Need to install the following packages:

create-vite@9.1.1

Ok to proceed? (y) y

> getexpress@1.0.0 npx

> create-vite my-react-app

|

◇ Select a framework:

| React

|

◇ Select a variant:

| JavaScript

|

◇ Which linter to use?

| ESLint

|

◇ Install with npm and start now?

| Yes

|

◇ Scaffolding project in C:\Users\CBIT\Desktop\getexpress\my-react-app...

|

◇ Installing dependencies with npm...

added 135 packages, and audited 136 packages in 58s

31 packages are looking for funding

run `npm fund` for details

found 0 vulnerabilities

|

◇ Starting dev server...

> my-react-app@0.0.0 dev

> vite

VITE v8.1.2 ready in 399 ms

→ Local: http://localhost:5173/

→ Network: use --host to expose

→ press h + enter to show help

PS C:\Users\CBIT\Desktop\REACT> CD REACT

up to date, audited 136 packages in 915ms

31 packages are looking for funding

run `npm fund` for details

found 0 vulnerabilities

```
PS C:\Users\CBIT\Desktop\REACT\REACT> npm.cmd run dev
```

```
> react@0.0.0 dev
```

```
> vite
```

VITE v8.1.2 ready in 207 ms

→ Local: <http://localhost:5173/>

→ Network: use --host to expose

→ press h + enter to show help

Terminate batch job (Y/N)?

```
PS C:\Users\CBIT\Desktop\REACT\REACT>
```

CREATE A NEW PROJECT “COMPONENT “ BY following above steps open “visual studio” click on open folder ,select the folder “COMPONENT” RUN THE BELOW COMMANDS MENTIONED IN VISUAL CODE BY NAVIGATING TERMINAL ----NEW TERMINAL

APP.JSX

```
function App() {
```

```
  return (
```

```
    <div>
```

```
      <h1>Welcome to React</h1>
```

```
      <h2>This is my first Functional Component</h2>
```

```
    </div>
```

```
  );
```

```
}
```

export default App;

press ctrlc to get the terminal console again

```
PS C:\Users\CBIT\Desktop\component> cd component
```

```
PS C:\Users\CBIT\Desktop\component\component> npm.cmd install
```

up to date, audited 136 packages in 949ms

31 packages are looking for funding

run `npm fund` for details

found 0 vulnerabilities

```
PS C:\Users\CBIT\Desktop\component\component> npm.cmd run dev
```

```
> component@0.0.0 dev
```

```
> vite
```

VITE v8.1.2 ready in 212 ms

→ Local: <http://localhost:5173/>

→ Network: use --host to expose

→ press h + enter to show help

CHANGE THE APP.JSX FILE

```
function App() {  
  
  const college = "CBIT";  
  const course = "React JS";  
  
  return (  
    <div>  
      <h1>{college}</h1>  
      <h2>{course}</h2>  
    </div>  
  );  
  
}
```

export default App;

press ctrlc to get the terminal console again

OPEN NEW TERMINAL IN VISUAL STUDIO

TYPE THE FOLLOWING COMMANDS

NOTE: IAM REPLACING THE APP.JSX ONLY PROJECT IS SAME

PS C:\Users\CBIT\Desktop\component> CD component

PS C:\Users\CBIT\Desktop\component\component> npm.cmd install

up to date, audited 136 packages in 1s

31 packages are looking for funding

run `npm fund` for details

found 0 vulnerabilities

PS C:\Users\CBIT\Desktop\component\component> npm.cmd run dev

> component@0.0.0 dev

> vite

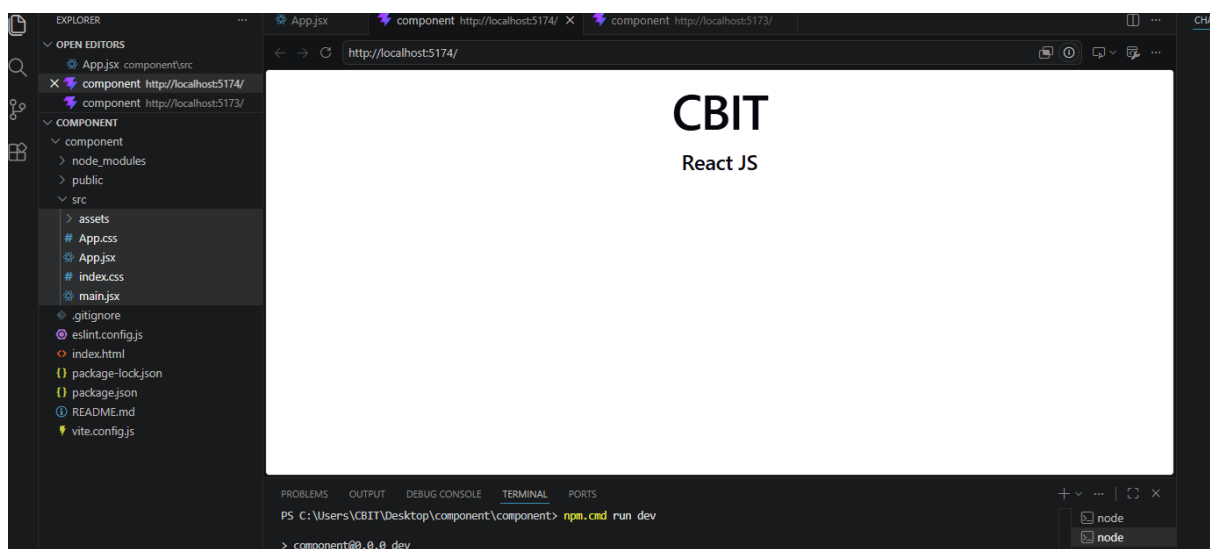
Port 5173 is in use, trying another one...

VITE v8.1.2 ready in 223 ms

→ Local: <http://localhost:5174/>

→ Network: use --host to expose

→ press h + enter to show help



Functional Component with Multiple Variables

```
function App() {  
  
  const name = "Rahul";  
  const age = 20;  
  const city = "Hyderabad";  
  
  return (  
    <div>  
      <h2>Student Details</h2>  
  
      <p>Name : {name}</p>  
  
      <p>Age : {age}</p>  
  
      <p>City : {city}</p>  
  
    </div>  
  );  
}
```

export default App;

run this commands in terminal again

```
PS C:\Users\CBIT\Desktop\component\component> npm.cmd run dev
```

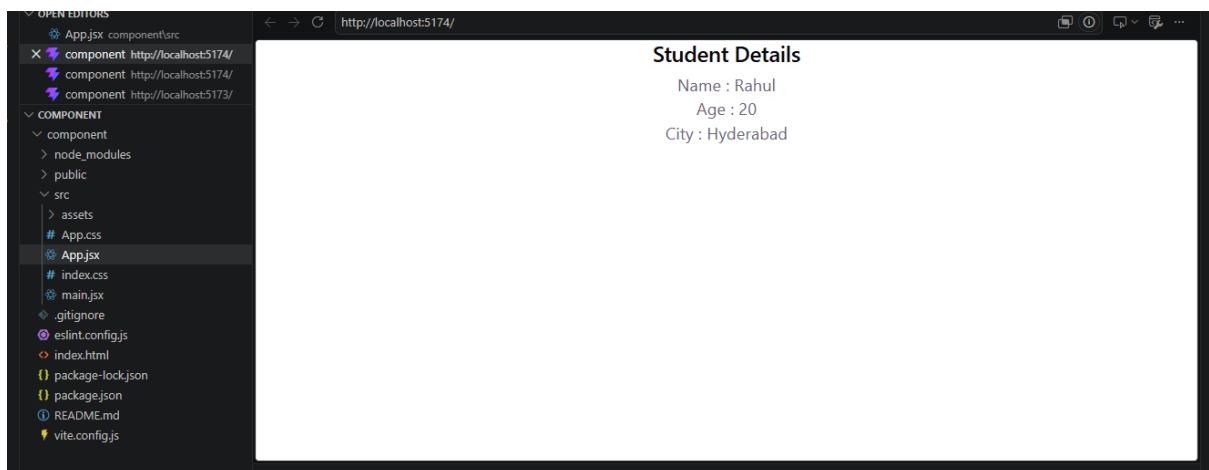
```
> component@0.0.0 dev
```

```
> vite
```

Port 5173 is in use, trying another one...

VITE v8.1.2 ready in 1999 ms

→ Local: <http://localhost:5174/>



Creating another Functional Component

```
function Student() {  
  
  return (  
    <h2>Welcome Student</h2>  
  );  
  
}
```

```
function App() {  
  
  return (  
    <div>  
  
      <h1>React Components</h1>  
  
      <Student />  
  
    </div>  
  );  
  
}
```

```
export default App;
```

run this commands in terminal again

```
PS C:\Users\CBIT\Desktop\component\component> npm.cmd run dev
```

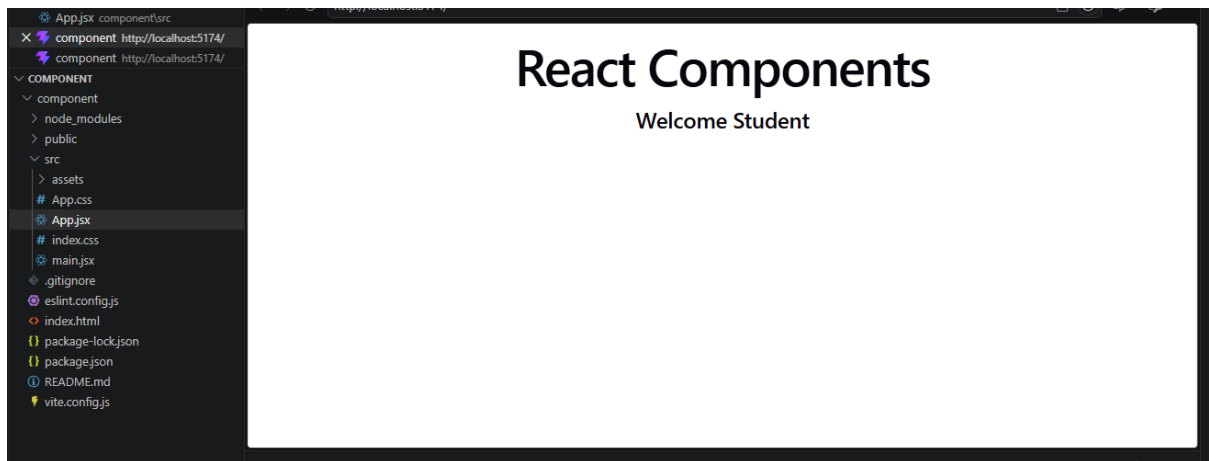
```
> component@0.0.0 dev
```

```
> vite
```

Port 5173 is in use, trying another one...

VITE v8.1.2 ready in 1999 ms

→ Local: <http://localhost:5174/>



Multiple Functional Components

```
function Header() {  
  return <h1>Header Component</h1>;  
}
```

```
function Content() {  
  return <h2>Content Component</h2>;  
}
```

```
function Footer() {  
  return <h3>Footer Component</h3>;  
}
```

```
function App() {  
  
  return (  

```

```
<div>
```

```
<Header />
```

```
<Content />
```

```
<Footer />
```

```
</div>
```

```
);
```

```
}
```

```
export default App;
```

press ctrlc to get the terminal console again

Creating Separate Component Files

Folder Structure

```
src
├── App.jsx
├── Header.jsx
├── Footer.jsx
└── Student.jsx
```

Header.jsx

```
function Header() {

  return (
    <h1>Welcome to React</h1>
  );
}
```

```
}
```

```
export default Header;
```

Student.jsx

```
function Student() {
```

```
  return (  
    <h2>Second Year B.Tech</h2>  
  );
```

```
}
```

```
export default Student;
```

Footer.jsx

```
function Footer() {
```

```
  return (  
    <h3>Thank You</h3>  
  );
```

```
}
```

```
export default Footer;
```

App.jsx

```
import Header from "./Header";  
import Student from "./Student";  
import Footer from "./Footer";
```

```
function App() {
```

```
  return (  
    <div>  
      <Header />  
      <Student />  
      <Footer />  
    </div>
```

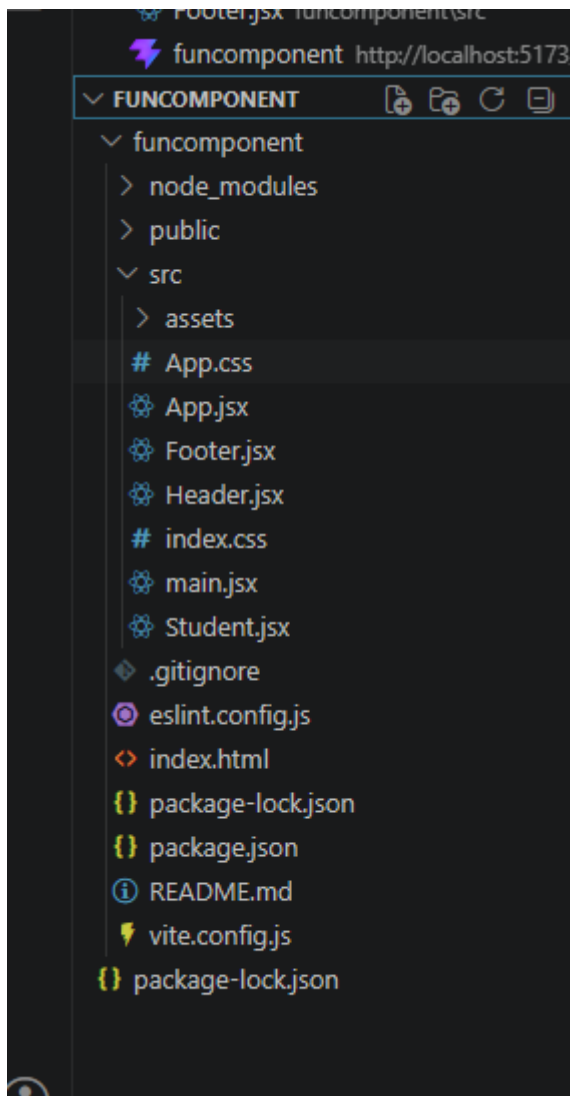
```
);  
}  
  
export default App;
```

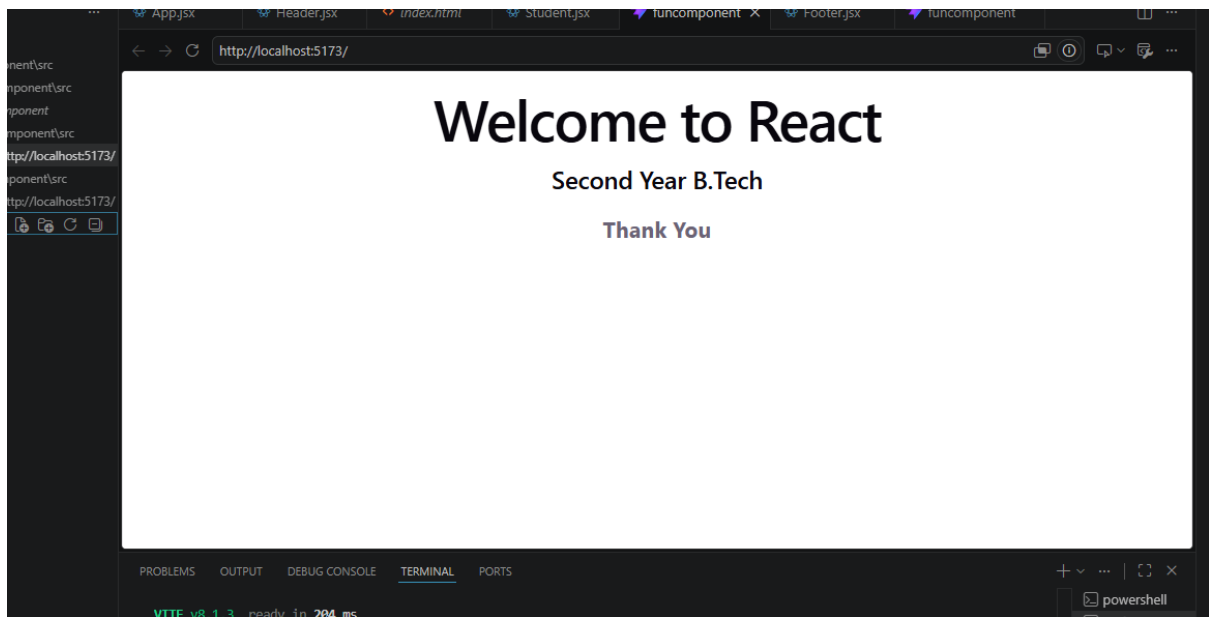
Output

Welcome to React

Second Year B.Tech

Thank You



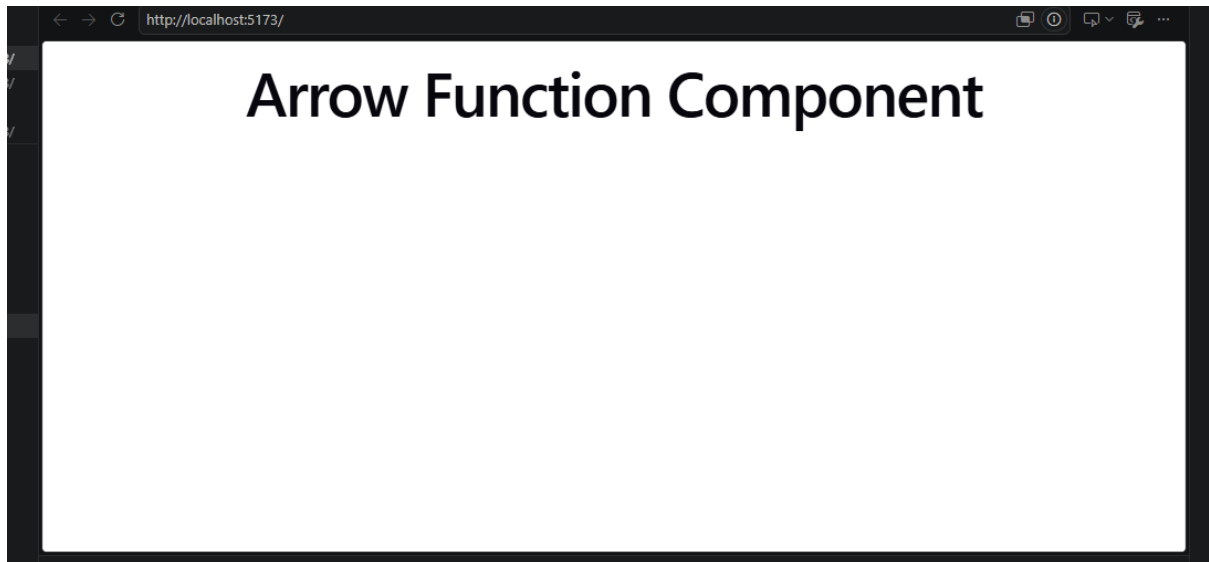


Functional Component Using Arrow Function

App.jsx

```
const App = () => {  
  
  return (  
  
    <div>  
  
      <h1>Arrow Function Component</h1>  
  
    </div>  
  
  );  
  
};
```

```
export default App;
```



Nested Functional Components

```
function College() {  
  return <h2>CBIT</h2>;  
}
```

```
function Student() {  
  return (  
    <div>  
      <College />  
      <h3>FALAK</h3>  
    </div>  
  );  
}
```

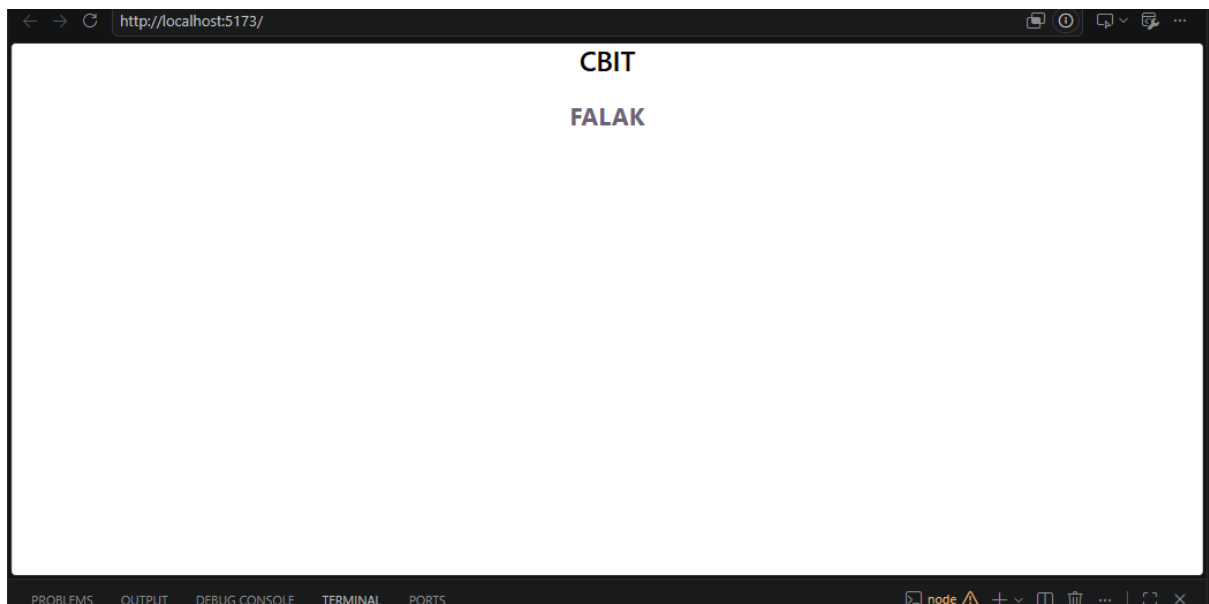
```
function App() {  
  return (  
    <Student />  
  );  
}
```

```
);  
}  
  
export default App;
```

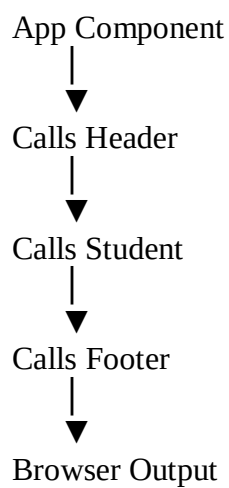
Output

CBIT

FALAK



Flow Diagram



Advantages of Functional Components

- Easy to create.
- Less code than class components.

Reusable.
Supports Hooks.
Better performance.
Easy to maintain.

Difference Between Functional and Class Components

Functional Component	Class Component
JavaScript function	JavaScript class
Easy to write	More code
Uses Hooks	Uses lifecycle methods
Recommended in modern React	Older approach
Lightweight	More complex

Actual DOM using HTML & JavaScript

Theory

The **Actual DOM** is the browser's Document Object Model. When JavaScript changes the DOM, the browser updates the actual HTML elements.

Flow Diagram

HTML Page



JavaScript



Modify DOM



Browser Repaints Entire Changed Section

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Actual DOM Example</title>
```

```
</head>
```

```
<body>

  <h2>Actual DOM Example</h2>

  <div id="displayTime"></div>

  <script>

    function updateTime() {

      let element = "";

      element += "<h1>Welcome to CBIT</h1>";

      element += "<h2>Current Time</h2>";

      element += "<h3>" + new Date().toLocaleTimeString() + "</h3>";

      document.getElementById("displayTime").innerHTML = element;

    }

  </body>
  updateTime();

  setInterval(updateTime,1000);

</script>

</html>
```



What happens internally?

Every Second

↓

JavaScript creates HTML

↓

innerHTML replaces DOM

↓

Browser redraws page
The DOM is modified directly.

\ Program 2: Virtual DOM (Basic React)

Theory

React creates a **Virtual DOM**, which is a lightweight copy of the Actual DOM.

React compares

Old Virtual DOM

with

New Virtual DOM

Only the changed element is updated.

Flow Diagram

React Component



Virtual DOM



Compare Previous DOM



Find Difference



Update Browser

App.jsx

```
function App() {  
  
  return (  
  
    <div>  
  
      <h1>Welcome to CBIT</h1>  
  
      <h2>Current Time</h2>  
  
      <h3>{new Date().toLocaleTimeString()}</h3>  
  
    </div>  
  
  );  
}  
  
export default App;
```

main.jsx

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import App from "./App";  
  
const root = ReactDOM.createRoot(document.getElementById("root"));  
  
function renderTime(){
```

```
    root.render(<App/>);  
  }  
  
  renderTime();  
  
  setInterval(renderTime,1000);
```

Run

npm run dev

Open

http://localhost:5173

Output

Welcome to CBIT

Current Time

10:45:12 AM

What happens?

Every Second

↓

New Virtual DOM

↓

Compare Old Virtual DOM

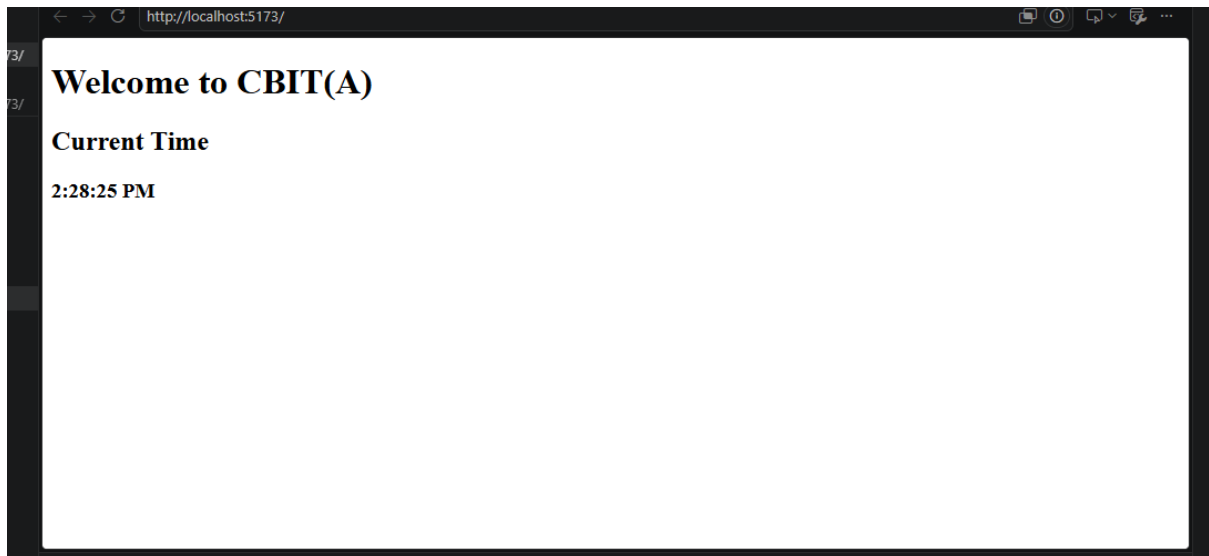
↓

Only Time Updated

↓

Browser Updated

No direct DOM manipulation.



Use this complete App.jsx code

Open **src/App.jsx** and replace everything with:

```
import { useState } from "react";

function App() {

  const [count, setCount] = useState(0);

  console.log("Component Rendered");

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>Virtual DOM Example</h1>

      <h2>Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}

export default App;
```

Output

You should see:

Virtual DOM Example

Count: 0

[Increment]

Clicking **Increment** changes:

Count: 1

Count: 2

Count: 3

.

Actual DOM Example with Increment Button

Save this file as **actualdom.html**.

```
<!DOCTYPE html>
<html>

<head>
  <title>Actual DOM - Counter Example</title>

  <style>
    body {
      font-family: Arial;
      text-align: center;
      margin-top: 100px;
    }

    button {
      padding: 10px 20px;
      font-size: 18px;
      cursor: pointer;
    }

    h2 {
      color: blue;
    }
  </style>

</head>

<body>

  <h1>Actual DOM Example</h1>

  <h2 id="count">0</h2>

  <button onclick="increment()">Increment</button>

  <script>

    // Variable to store count
    let count = 0;
```

```
// Function to increment count
function increment() {

    count++;

    // Directly updating the Actual DOM
    document.getElementById("count").innerHTML = count;

    console.log("Actual DOM Updated");

}

</script>

</body>

</html>
```

Output

Initially:

Actual DOM Example

0

[Increment]

After clicking the button once:

Actual DOM Example

1

[Increment]

After clicking again:

Actual DOM Example

2

[Increment]

How to Execute

Step 1

Open **Visual Studio Code**.

Step 2

Create a file named:

actualdom.html

Step 3

Paste the above code.

Step 4

Save the file.

Step 5

Right-click the file and select:

Open with Live Server

(or simply double-click the HTML file to open it in a browser).

How It Works

User Clicks Increment Button



JavaScript Function increment()



count = count + 1



document.getElementById("count")



innerHTML = count



Browser Updates the Actual DOM

In this program, every time the user clicks **Increment**:

JavaScript directly accesses the DOM using:

```
document.getElementById("count")
```

It changes the HTML content using:

```
innerHTML
```

The browser immediately updates the displayed value.

This is called **Actual DOM manipulation** because JavaScript directly modifies the browser's DOM.

```
import { useState } from "react";
```

```
function App() {  
  
  const [count, setCount] = useState(0);  
  
  console.log("Component Rendered");  
  
  return (  
    <div style={{ textAlign: "center", marginTop: "50px" }}>  
      <h1>Virtual DOM Example</h1>  
  
      <h2>Count: {count}</h2>  
  
      <button onClick={() => setCount(count + 1)}>  
        Increment  
      </button>  
    </div>  
  );  
}
```

```
export default App;
```

VIRTUAL DOM

```
import { useState } from "react";
```

```
function App() {  
  
  const [count, setCount] = useState(0);
```

```

console.log("Component Rendered");

return (
  <div style={{ textAlign: "center", marginTop: "50px" }}>
    <h1>Virtual DOM Example</h1>

    <h2>Count: {count}</h2>

    <button onClick={() => setCount(count + 1)}>
      Increment
    </button>
  </div>
);
}

```

export default App;

Instead of just showing the counter, **open the browser's Developer Tools:**

1. Open the HTML page.
2. Press **F12**.
3. Go to the **Elements** tab.
4. Click **Increment**.

Then open the React page:

1. Press **F12**.
2. Click **Increment**.

In **JavaScript**, **you** modify the DOM using `document.getElementById()`.

In **React**, **you never touch the DOM**. You update the state (`setCount()`), and **React updates the DOM for you** using the Virtual DOM.

```
import React, { useState } from "react";

function App() {

  const [count, setCount] = useState(0);


  return (

    <div>

      <h1>Hello Students</h1>

      <p>Count: {count}</p>

      <button onClick={() => setCount(count + 1)}>

        Increment

      </button>

    </div>

  );
}

export default App;
```

Now connect the code to your diagram:
VIRTUAL DOM

State Change



setCount(count + 1)

↓

New Virtual DOM

↓

React creates a new UI representation

↓

Compare with Old Virtual DOM

↓

React's reconciliation process

↓

Find Changes

↓

"Count: 0" → "Count: 1"

↓

Update Actual DOM

↓

<p>Count: 1</p>

↓

UI/Screen

Explain each line

1. State

```
const [count, setCount] = useState(0);
```

Initial state is:

```
count = 0
```

2. User clicks the button

```
<button onClick={() => setCount(count + 1)}>
```

setCount() changes the state:
0 → 1

3. React creates a new Virtual DOM representation
The previous UI was conceptually:
<p>Count: 0</p>

The new UI is:
<p>Count: 1</p>

4. React compares the previous and new representations
Old: <p>Count: 0</p>
New: <p>Count: 1</p>

React determines that the relevant text needs to change.
5. Actual DOM is updated
Conceptually, the browser DOM changes from:
<p>Count: 0</p>

to:
<p>Count: 1</p>

6. User sees
Hello Students

Count: 1

[Increment]

“React creates a new Virtual DOM representation, compares it with the previous one during reconciliation, determines the necessary DOM changes, and commits those changes to the Actual DOM.”

Demonstration the Actual DOM vs Virtual DOM difference
// Actual DOM manipulation

```
document.getElementById("count").innerText = 1;
```

versus React:

```
const [count, setCount] = useState(0);
```

```
<button onClick={() => setCount(count + 1)}>
```

Increment

```
</button>
```

```
<p>{count}</p>
```

This makes the distinction very clear:

JavaScript

React

getElementById()

setCount()



Actual DOM directly

State change



React reconciliation



Actual DOM update

What is Reconciliation in React?

Reconciliation is the process React uses to compare the previous Virtual DOM with the new Virtual DOM and determine what changes need to be made to the Actual DOM.

Simple flow

State changes



New Virtual DOM



Reconciliation



Compare old Virtual DOM

with new Virtual DOM



Identify changes



Update Actual DOM



Browser UI

Simple example

Suppose initially we have:

`<p>Count: 0</p>`

After clicking a button:

`<p>Count: 1</p>`

React conceptually has:

OLD Virtual DOM NEW Virtual DOM

`<p>Count: 0</p>` $\rightarrow$ `<p>Count: 1</p>`

During reconciliation, React determines:

`"Count: 0"  $\rightarrow$  "Count: 1"`

So the necessary DOM update is made.

Code Example

```
import { useState } from "react";
```

```
function Counter() {
```

```
  const [count, setCount] = useState(0);
```

```
return (  
  <div>  
    <h1>Counter</h1>  
  
    <p>Count: {count}</p>  
  
    <button onClick={() => setCount(count + 1)}>  
      Increment  
    </button>  
  </div>  
);  
}
```

```
export default Counter;
```

When the button is clicked:
`setCount(count + 1);`

What happens?

1. State changes

↓

count: 0 → 1

2. React renders again

↓

3. New Virtual DOM is created



4. Reconciliation



Old: `<p>Count: 0</p>`

New: `<p>Count: 1</p>`

5. React determines what changed



6. Actual DOM is updated



`<p>Count: 1</p>`

7. Browser displays the result

Introduction

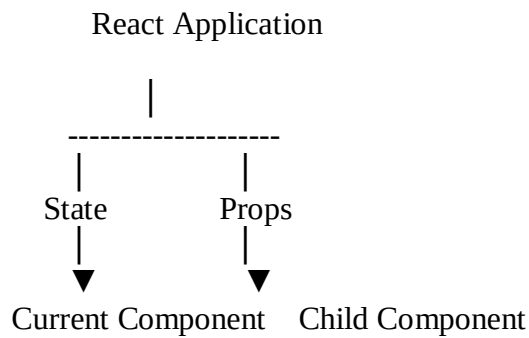
React applications are built using **components**. Every component may contain data that needs to be displayed or updated.

React provides two important concepts to manage this data:

1. **State**
2. **Props (Properties)**

These are the foundation of React applications.

Data Flow in React



What is State?

Definition

State is a built-in object that stores data belonging to a component. This data can change over time, and whenever it changes, React automatically updates the user interface.

Key Points

- State is local to a component.
- State is mutable (can change).
- Updating state re-renders the component.
- State is created using the **useState()** hook.

Characteristics of State

- Local to a component
- Stores dynamic data
- Can be modified
- Automatically updates the UI
- Managed using Hooks

State and Props in React

In React, **state** and **props** are two important concepts used to manage and pass data between components.

1. What are Props?

Props (short for **properties**) are used to **pass data from a parent component to a child component**.

Key points about Props

- Props are passed from **parent** → **child**.
- Props are **read-only** inside the child component.
- A child component should **not modify its props**.

- Props make components reusable.
- Props can contain strings, numbers, arrays, objects, functions, etc.

Example of Props

```
function Student(props) {
  return <h2>Hello, {props.name}!</h2>;
}

function App() {
  return (
    <div>
      <Student name="Rahul" />
      <Student name="Priya" />
    </div>
  );
}
```

export default App;

Here:

```
<Student name="Rahul" />
```

name="Rahul" is a **prop**.

The parent (App) passes the name prop to the child (Student).

The child receives it through:

```
props.name
```

Using destructuring

Instead of:

```
function Student(props) {
  return <h2>Hello, {props.name}!</h2>;
}
```

You can write:

```
function Student({ name }) {
  return <h2>Hello, {name}!</h2>;
}
```

This is generally cleaner and commonly used.

What is State?

State is data that belongs to a component and can **change over time**.

When state changes, React **re-renders the component** so that the UI displays the updated data.

For functional components, we usually manage state using the `useState` Hook.

Syntax

```
const [state, setState] = useState(initialValue);
```

For example:

```
const [count, setCount] = useState(0);
```

Here:

- count → current state value
- setCount → function used to update the state
- 0 → initial value

3. Example of State

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}

export default Counter;
```

Initially:

Count: 0

When you click **Increase**:

Count: 1

Click again:

Count: 2

The value of count changes, so React re-renders the component.

State vs Props

Props	State
Data passed from parent to child	Data managed by the component
Read-only	Can be updated
Used to communicate between components	Used to manage changing data
Controlled by parent	Controlled by the component itself
Changes to props come from the parent	Changes using a state setter
Example: name="Rahul"	Example: count

Props = information given to a component

State = information owned and changed by a component

Example Using Both State and Props

Example shows how **state** and **props** work together

```
import { useState } from "react";

function User({ name, count }) {
  return (
    <div>
      <h2>Hello {name}</h2>
      <p>Count: {count}</p>
    </div>
  );
}

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <User name="Rahul" count={count} />

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}
```

export default App;

What is happening?

The App component owns the state:

```
const [count, setCount] = useState(0);
```

Then App passes that state to User as a prop:

```
<User name="Rahul" count={count} />
```

So:

```
App
|
| props
↓
User
```

name is a normal prop:

```
name="Rahul"
```

count is also a prop, but its value comes from the parent's state:

```
count={count}
```

When the button is clicked:

```
setCount(count + 1)
```

the parent's state changes.

React re-renders App, and the new count is passed to User.

```
count = 0
↓
button clicked
↓
setCount(1)
↓
App re-renders
↓
<User count={1} />
↓
User displays 1
```

6. Can a Child Change Parent State?

Yes, but not directly.

The child cannot directly modify the parent's state.

Instead, the parent can pass a **function as a prop** to the child.

Example:

```
import { useState } from "react";

function Child({ increase }) {
  return (
    <button onClick={increase}>
      Increase Count
    </button>
  );
}

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>

      <Child increase={() => setCount(count + 1)} />
    </div>
  );
}

export default App;
```

Here the parent passes a function:

```
increase={() => setCount(count + 1)}
```

to the child.

The child calls it:

```
onClick={increase}
```

This is a common React pattern called **passing a callback/function through props**.

Props → data received by component

State → data that component manages and changes

Props are used to pass data between components, while state is used to store and manage data that can change inside a component.

useState and useEffect

useState **and** useEffect **in React**

useState and useEffect are two of the most commonly used **React Hooks**.

- useState → used to store and update data in a component.
- useEffect → used to perform side effects after a component renders.

useState

useState allows a functional component to **remember information between renders**.

Syntax

```
const [state, setState] = useState(initialValue);
```

For example:

```
const [count, setCount] = useState(0);
```

Here:

- count → current state value
- setCount → function used to update count
- 0 → initial value

When you call setCount(), React updates the state and **re-renders the component**.

Example

```
import { useState } from "react";
```

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <h2>Count: {count}</h2>  
  
      <button onClick={() => setCount(count + 1)}>  
        Increase  
      </button>  
    </div>  
  );  
}
```

```
export default Counter;
```

How it works

Initially:

```
count = 0
```

After clicking the button:

```
setCount(1)
```

↓

```
count = 1
```

↓

React re-renders

↓

UI displays 1

If you click again:

```
count = 2
```

Important points about `useState`

- State stores data that can change.
- Updating state causes a re-render.
- Don't change state directly.
- Use the setter function to update it.

```
count = count + 1;
```

```
setCount(count + 1);
```

`useEffect`

`useEffect` is used when your component needs to perform a **side effect**.

A side effect is something that happens outside the normal process of calculating/rendering the UI.

Common examples include:

- Fetching data from an API
- Changing the document title
- Setting up a timer
- Adding event listeners
- Subscribing/unsubscribing to something
- Working with browser APIs

Syntax

```
useEffect(() => {  
  // side effect code  
}, [dependencies]);
```

3. Simple `useEffect` Example

```
import { useState, useEffect } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);

  return (
    <div>
      <h2>Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}
```

`export default Counter;`

When the component loads:

`count = 0`

The browser title becomes:

`Count: 0`

When the button is clicked:

`count = 1`

The effect runs again:

`document.title = "Count: 1"`

4. Dependency Array

The second argument of `useEffect` is called the **dependency array**.

It controls **when the effect runs**.

Case 1: No dependency array

```
useEffect(() => {
  console.log("Effect runs");
});
```

The effect runs **after every render**.

```
Render
  ↓
Effect
  ↓
State changes
  ↓
Render
  ↓
Effect
```

Case 2: Empty dependency array

```
useEffect(() => {  
  console.log("Effect runs");  
}, []);
```

The effect runs **once after the initial render**.

This is commonly used for things such as initial data loading.

Component renders

↓
Effect runs

↓
No dependency changes

↓
Effect doesn't run again

Case 3: Dependency [count]

```
useEffect(() => {  
  console.log("Count changed");  
}, [count]);
```

The effect runs after the initial render and whenever `count` changes.

For example:

```
count = 0 → Effect  
count = 1 → Effect  
count = 2 → Effect  
count = 3 → Effect
```

But if some unrelated state changes, the effect doesn't re-run just because of that.

Example

useState useEffect

A common example is a **counter with a document title**:

```
import { useState, useEffect } from "react";  
  
function App() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    document.title = `You clicked ${count} times`;  
  }, [count]);  
  
  return (  
    <div>  
      <h1>You clicked {count} times</h1>  
  
      <button onClick={() => setCount(count + 1)}>  
        Click Me  
      </button>  
    </div>  
  );  
}  
  
export default App;
```

Flow

```
useState
  ↓
count = 0
  ↓
Component renders
  ↓
useEffect runs
  ↓
Page title = "You clicked 0 times"
  ↓
Button clicked
  ↓
setCount(1)
  ↓
Component re-renders
  ↓
useEffect detects count changed
  ↓
Page title = "You clicked 1 times"
```

6. API Fetching Example

One of the most common uses of `useEffect` is fetching data.

```
import { useState, useEffect } from "react";

function Users() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => response.json())
      .then((data) => {
        setUsers(data);
      });
  }, []);

  return (
    <div>
      <h2>Users</h2>

      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}
```

```
export default Users;
```

Here:

```
const [users, setUsers] = useState([]);
```

stores the users.

And:

```
useEffect(() => {
  fetch(...)
}, []);
```

fetches the data when the component initially loads.

When the data arrives:

```
setUsers(data);
```

updates the state, causing React to render the users.

Cleanup Function in useEffect

Some effects need to be **cleaned up** when a component is removed or before the effect runs again.

For example, a timer:

```
import { useState, useEffect } from "react";

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const timer = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);

    return () => {
      clearInterval(timer);
    };
  }, []);

  return <h2>Seconds: {seconds}</h2>;
}
```

```
export default Timer;
```

The cleanup function is:

```
return () => {
  clearInterval(timer);
};
```

It prevents the timer from continuing to run after the component is removed.

useState **Vs** useEffect

useState	useEffect
Stores data	Performs side effects
Used for changing values	Used for operations after rendering
Returns state and setter	Runs a function

useState	useEffect
Updating state causes re-render	Can respond to state/prop changes
Example: counter, form input	Example: API call, timer
useState(0)	useEffect(() => {}, [])

Example together

```
const [count, setCount] = useState(0);

useEffect(() => {
  console.log("Count changed:", count);
}, [count]);
```

Here, useState **manages the data**, while useEffect **performs an action when that data changes**.

Syntax of State

```
const [stateVariable, setStateVariable] = useState(initialValue);
```

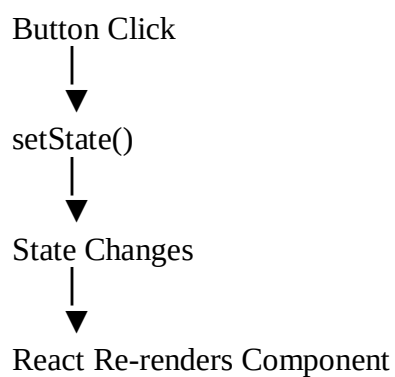
Example:

```
const [count, setCount] = useState(0);
```

Where:

count → current value
 setCount() → updates the value
 0 → initial value

State Flow Diagram





Browser Displays Updated Data

Example 1: Counter Using State

App.jsx

```
import { useState } from "react";

function App() {

  const [count, setCount] = useState(0);

  return (

    <div style={{textAlign:"center"}}>

      <h1>Counter Application</h1>

      <h2>{count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

    </div>

  );
}
```

```
export default App;
```

Output

Counter Application

0

[Increment]
After clicking:

1

Counter Application

1

Increment

EXAMPLE -2

Step 1: Create a Vite React Project

Open the terminal in VS Code and run:

```
npm.cmd create vite@latest my-react-app
```

Choose:

Framework: React

Variant: JavaScript

Step 2: Go to the Project Folder

```
cd my-react-app
```

Step 3: Install Dependencies

```
npm.cmd install
```

Step 4: Open the Project

code .

Step 5: Replace App.jsx

Open:

```
my-react-app
├──
└── src
    └── App.jsx
```

Delete everything and paste your React code.

Example:

```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>Counter App</h1>

      <h2>{count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrement
      </button>

      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}

export default App;
```

Step 6: Check main.jsx

Open:

```
src/main.jsx
```

It should look like this:

```
import React from "react";
import ReactDOM from "react-dom/client";
```

```
import App from "./App.jsx";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Step 7: Run the Project

In the terminal, run:

```
npm run dev
```

You will see output similar to:

```
VITE v7.x.x ready in 500 ms
```

```
→ Local: http://localhost:5173/
```

```
→ Network: http://192.168.1.5:5173/
```

Step 8: Open the Browser

Open:

```
http://localhost:5173/
```

You will see:

Counter App

0

Increment	Decrement	Reset
-----------	-----------	-------

Counter App

0

[Increment] [Decrement] [Reset]

Folder Structure

```
my-react-app
├── node_modules
├── public
├── src
│   ├── App.jsx
│   ├── main.jsx
│   ├── index.css
│   └── assets
├── package.json
├── vite.config.js
└── ...
```

Commands Summary

```
npm create vite@latest my-react-app
cd my-react-app
npm install
npm run dev
```

Example : Change Text using State

```
import React, { useState } from "react";

function App() {

  const [message, setMessage] = useState("Welcome");

  return (
    <div style={{ textAlign: "center" }}>
      <h1>{message}</h1>

      <button onClick={() => setMessage("Hello React!")}>
        Change Text
      </button>
    </div>
  );
}
```

```
export default App;
```

Welcome

Change Text

after click on change text

Hello React!

Change Text

Example : Show/Hide Text

```
import React, { useState } from "react";
```

```
function App() {
```

```
  const [show, setShow] = useState(true);
```

```
  return (
```

```
    <div style={{ textAlign: "center" }}>
```

```
      <button onClick={() => setShow(!show)}>
```

```
        Toggle
```

```
      </button>
```

```
      {show && <h2>React State Example</h2>}
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```

Toggle

React State Example

click on toggle

Toggle

Example : Input Box using State

```
import React, { useState } from "react";

function App() {

  const [name, setName] = useState("");

  return (
    <div style={{ textAlign: "center" }}>

      <h2>Hello {name}</h2>

      <input
        type="text"
        placeholder="Enter your name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

    </div>
  );
}

export default App;
```



Hello FALAK

FALAK

EXAMPLE -ARRAY STATE

app.jsx

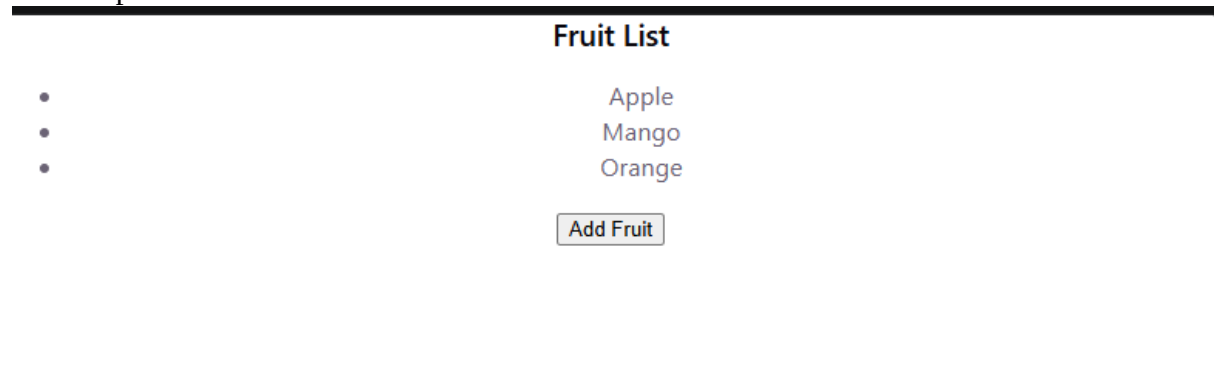
```
import React, { useState } from "react";
```

```
function App() {  
  
  const [fruits, setFruits] = useState([  
    "Apple",  
    "Mango",  
    "Orange"  
  ]);  
  
  const addFruit = () => {  
    setFruits([...fruits, "Banana"]);  
  };  
  
  return (  
    <div>  
  
      <h2>Fruit List</h2>  
  
      <ul>  
        {fruits.map((fruit, index) => (  
          <li key={index}>{fruit}</li>  
        ))}  
      </ul>  
  
      <button onClick={addFruit}>  
        Add Fruit  
      </button>  
  
    </div>  
  );  
}
```

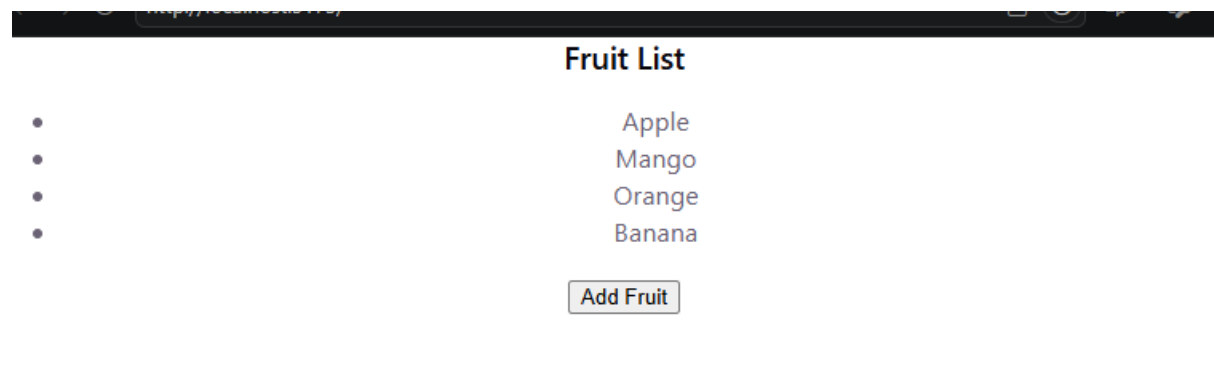
```
export default App;
```

OUTPUT:

OPEN TERMINAL
TYPE npm.cmd run dev



click on add fruit



EXAMPLE - STUDENT DETAILS

CHANGE THE CODE OF APP.JSX

```
import React, { useState } from "react";
```

```
function App() {
```

```
  const [employee] = useState({
    id: 101,
    name: "John",
    designation: "Software Engineer"
  });
```

```
  return (
```

```
    <div>
```

```
      <h2>Employee Details</h2>
```

```
<p>ID : {employee.id}</p>
<p>Name : {employee.name}</p>
<p>Designation : {employee.designation}</p>

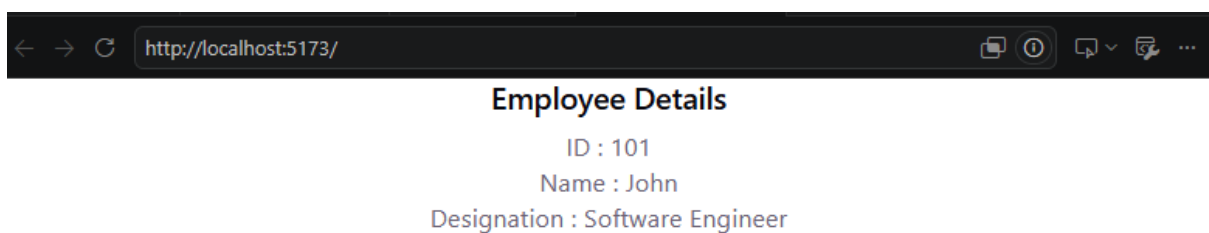
</div>
);
}

export default App;
```

OUTPUT

TYPE IN TERMINAL

NPM.CMD RUN DEV(WE ARE USING THE EXISTING PROJECT EMP)



EXAMPLE 4:STUDENT DETAIL FORM

app.jsx

```
import React, { useState } from "react";
```

```
function App() {

  const [student, setStudent] = useState({

    name: "",

    course: ""

  });

  return (

    <div>

      <input

        type="text"

        placeholder="Name"

        onChange={(e) =>

          setStudent({ ...student, name: e.target.value })

        }

      />

      <br /><br />

      <input

        type="text"

        placeholder="Course"

        onChange={(e) =>

          setStudent({ ...student, course: e.target.value })

        }

      />
```

```

    <h3>Name: {student.name}</h3>

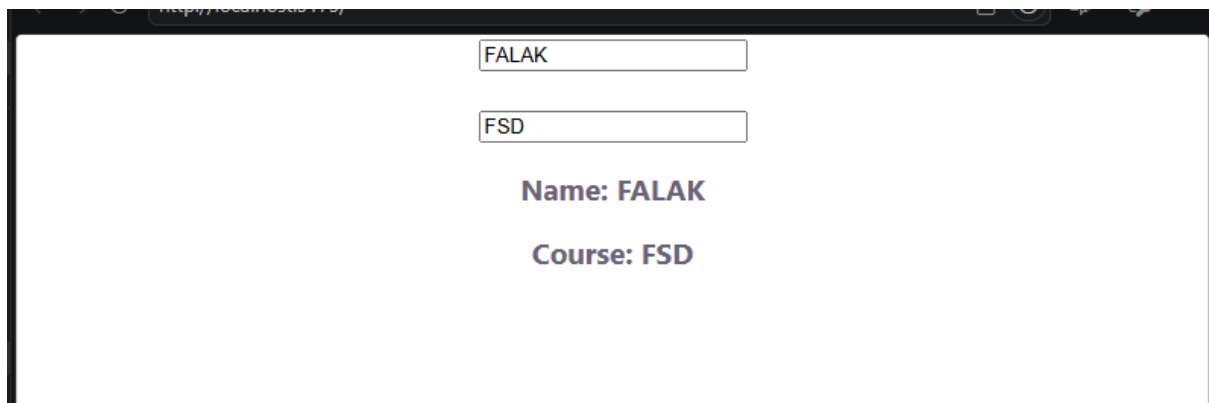
    <h3>Course: {student.course}</h3>

  </div>

);
}

export default App;

```



Employee Table with Add Button

app.jsx

```

import React, { useState } from "react";

function App() {

  const [employees, setEmployees] = useState([
    { id: 1, name: "John", designation: "SE" },

```

```
{ id: 2, name: "Tom", designation: "SSE" },  
{ id: 3, name: "Kevin", designation: "TL" }  
]);
```

```
const addEmployee = () => {  
  setEmployees([  
    ...employees,  
    {  
      id: 4,  
      name: "Sam",  
      designation: "Manager"  
    }  
  ]);  
};
```

```
return (  
  <div>
```

```
    <table border="1" cellPadding="10">
```

```
      <thead>
```

```
        <tr>
```

```
          <th>ID</th>
```

```
          <th>Name</th>
```

```
          <th>Designation</th>
```

```
        </tr>
```

```
      </thead>
```

```

  {employees.map((emp) => (
    <tr key={emp.id}>
      <td>{emp.id}</td>
      <td>{emp.name}</td>
      <td>{emp.designation}</td>
    </tr>
  ))}
</tbody>

</table>

<br />

<button onClick={addEmployee}>
  Add Employee
</button>

</div>

);
}

```

```
export default App;
```

useState Syntax

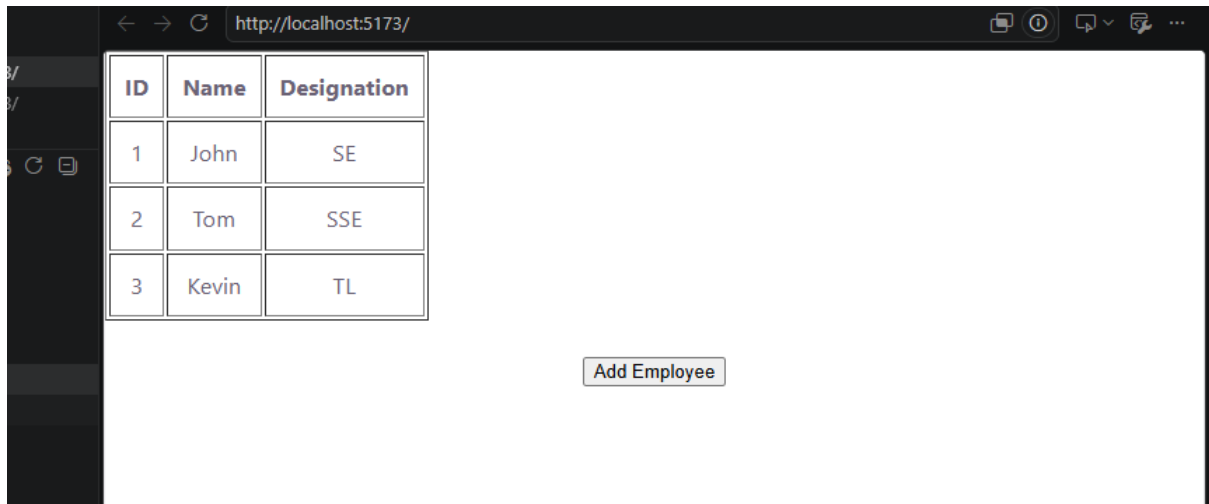
```
const [state, setState] = useState(initialValue);
```

- state → Stores the current value.
- setState → Updates the value.
- initialValue → The initial state (number, string, array, object, etc.).

OUTPUT

TYPE IN TERMINAL

npm.cmd run dev (using existing project emp so no need to type npm.cmd install)



AFTER CLICKING ON

‘Add Employee’ button



What are Props?

Definition

Props (Properties) are used to pass data from a **Parent Component** to a **Child Component**.

Props make components reusable.

Characteristics of Props

- Read-only
 - Passed from parent to child
 - Child cannot modify props
 - Used for communication between components
-

Real-Life Example

Teacher writes marks on a report card.

Student receives them.

The student cannot change them.

Teacher → Parent Component

Marks → Props

Student → Child Component

Props Flow Diagram

Parent Component

|

Passes Props

|



Child Component

|

Displays Data

Syntax

Parent Component

```
<Student name="Rahul" age="20" />
```

Child Component

```
function Student(props){  
  
  return(  
  
    <h2>{props.name}</h2>  
  
  );  
  
}
```

Example 2: Props Example

Student.jsx

```
function Student(props){  
  
  return(  
  
    <div>  
  
      <h2>Name : {props.name}</h2>  
  
      <h2>Age : {props.age}</h2>  
  
      <h2>College : {props.college}</h2>  
  
    </div>  
  
  );  
  
}  
  
export default Student;
```

App.jsx

```
import Student from "./Student";  
  
function App(){  
  
  return(  
  
    <div>  
  
      <Student
```

```
        name="DEEPAK"
        age="20"
        college="CBIT"
    />

</div>

);

}

export default App;
```

Output

```
Name : DEEPAK
Age : 20
College : CBIT
```

```
Name : Rahul

Age : 20

College : CBIT
```

Example 3: Reusable Component

App.jsx

```
import Student from "./Student";

function App(){

    return(

        <div>
```

```
    <Student
      name="Rahul"
      age="20"
      college="CBIT"
    />

    <Student
      name="Priya"
      age="19"
      college="MGIT"
    />

  </div>

);

}
```

export default App;

Output

type in terminal

npm.cmd run dev



Name : Rahul
Age : 20
College : CBIT
Name : Priya
Age : 19
College : MGIT

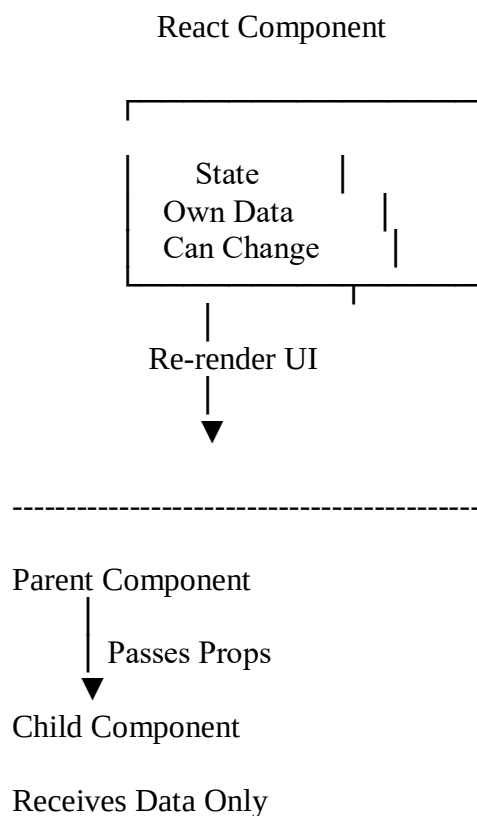
Name : Rahul
Age : 20
College : CBIT

Name : Priya
Age : 19
College : MGIT

State vs Props

State	Props
Stores data inside a component	Passes data from parent to child
Mutable (can change)	Immutable (read-only)
Managed by the component	Passed by the parent
Uses useState()	Uses JSX attributes
Triggers re-render when updated	Does not change in the child

State vs Props Diagram



State vs Props Example

Using State

```
const [name, setName] = useState("Rahul");
```

The component can change the value:

```
setName("Amit");
```

Using Props

Parent Component

```
<Student name="Rahul"/>
```

Child Component

```
function Student(props){  
  
  return <h2>{props.name}</h2>;  
  
}
```

The child component **cannot** change props.name.

When to Use State?

Use State when:

- Counter
- Calculator
- Login Form
- Shopping Cart
- Todo List
- Online Quiz
- Timer
- Digital Clock

When to Use Props?

Use Props when:

- Student Details
- Employee Information
- Product Details
- User Profile
- Displaying Data
- Parent-to-Child Communication

Advantages of State

- Stores dynamic data
- Updates the UI automatically
- Makes applications interactive

- Easy to manage using Hooks

Advantages of Props

- Makes components reusable
- Enables parent-child communication
- Improves code readability
- Encourages modular design

this is a **Props with Destructuring** example. Here's how to execute it step by step.

Step 1: Open Your React Project

Open your React project in VS Code.

Example:

my-react-app

Step 2: Create Greeting.jsx

Inside the **src** folder, create a new file named:

Greeting.jsx

Paste the following code:

```
const Greeting = ({ name }) => {  
  return (  
    <h2>Welcome, {name}!</h2>  
  );  
};
```

```
export default Greeting;
```

Explanation

```
({ name })
```

is called **object destructuring**.

Instead of writing:

```
function Greeting(props) {  
  return <h2>Welcome, {props.name}!</h2>;  
}
```

we directly write:

```
function Greeting({ name }) {  
  return <h2>Welcome, {name}!</h2>;  
}
```

Both are **equivalent**.

Step 3: Open App.jsx

Replace the existing code with:

```
import Greeting from "./Greeting";
```

```
function App() {
```

```
  return (
```

```
    <div>
```

```
      <h1>Props using Destructuring</h1>
```

```
      <Greeting name="Lakshmi" />
```

```
<Greeting name="React Student" />
```

```
</div>
```

```
);
```

```
}
```

```
export default App;
```

Step 4: Folder Structure

Your project should look like this:

my-react-app

|

|— src

| |— App.jsx

| |— Greeting.jsx

| |— main.jsx

|

|— package.json

|— index.html

Step 5: Start the React Server

Open the terminal in VS Code.

Run:

npm run dev

Step 6: Open the Browser

Go to:

http://localhost:5173

Output

Props using Destructuring

Welcome, Lakshmi!

Welcome, React Student!

Data Flow

App.jsx (Parent)

<Greeting name="Lakshmi"/>

|

Passes Props

|



Greeting.jsx (Child)

```
const Greeting = ({name})
```

|



Welcome, Lakshmi!

Interview Question

What is Destructuring in React?

Answer:

Destructuring is a JavaScript feature used to extract values from an object or array. In React, it is commonly used to directly access props without writing props.propertyName.

Without destructuring:

```
function Greeting(props) {  
  return <h2>{props.name}</h2>;  
}
```

With destructuring:

```
function Greeting({ name }) {  
  return <h2>{name}</h2>;  
}
```

Both produce the same output, but destructuring makes the code shorter, cleaner, and easier to read.

React Hooks (useState and useEffect)

Learning Objectives

After completing this topic, students will be able to:

- Understand React Hooks.

- Explain the useState Hook.
 - Explain the useEffect Hook.
 - Create interactive React applications using Hooks.
 - Differentiate between useState and useEffect.
-

What are React Hooks?

Definition

Hooks are special functions introduced in **React 16.8** that allow **functional components** to use React features such as **state** and **lifecycle methods** without writing class components.

Common Hooks

- useState()
- useEffect()
- useContext()
- useRef()
- useMemo()
- useCallback()

The two most commonly used hooks are:

- useState()
 - useEffect()
-

useState() Hook

Definition

The useState() Hook is used to **create and manage state** in a functional component.

Whenever the state changes, React automatically updates the user interface.

Syntax

```
const [state, setState] = useState(initialValue);
```

Explanation

- state → Current value
 - setState → Function to update the value
 - initialValue → Initial state value
-

Flow Diagram

User Click

|



setState()

|



State Updated

|



Component Re-renders

|



Updated UI

Example 1: Counter using useState

App.jsx

```
import { useState } from "react";
```

```
function App() {
```

```
  const [count, setCount] = useState(0);
```

```
return (  
  <div style={{ textAlign: "center" }}>  
    <h1>Counter Application</h1>  
  
    <h2>{count}</h2>  
  
    <button onClick={() => setCount(count + 1)}>  
      Increment  
    </button>  
  </div>  
);  
}
```

export default App;

Output

Counter Application

0 [Increment]

After clicking:

1

Counter Application

3

Increment

Example 2: Toggle Button

```
import { useState } from "react";

function App() {

  const [status, setStatus] = useState("OFF");

  return (

    <div>

      <h2>Switch is {status}</h2>

      <button
        onClick={() =>
          setStatus(status === "OFF" ? "ON" : "OFF")
        }
      >
        Toggle
      </button>

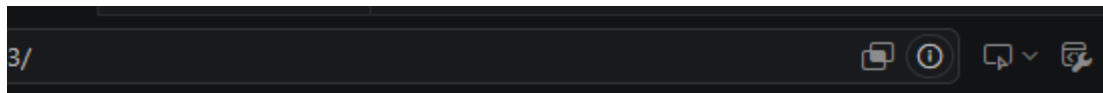
    </div>

  );
}

export default App;
```

Switch is ON

Toggle



Switch is OFF

Toggle

useEffect() Hook

Definition

The useEffect() Hook is used to perform **side effects** in React components.

Side Effects include:

- Fetching data from an API
- Updating the document title
- Timers
- Event listeners
- Reading browser storage

Syntax

```
useEffect(() => {
```

```
// Code
```

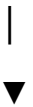
```
}, []);
```

Flow Diagram

Component Loads



useEffect()



Execute Side Effect

What is a Hook?

A **Hook** is a special function in React that allows functional components to use React features such as **state** and **lifecycle methods**.

The most commonly used Hook is:

useState()

What is useState()?

Definition

The **useState() Hook** is used to create and manage **state** (data) in a functional component.

Whenever the state changes, React automatically updates the user interface (UI).

Why do we use useState()?

Without useState(), the values displayed in a component cannot change dynamically.

Using `useState()` allows us to:

- Store data
- Update data
- Automatically refresh the UI

Syntax

```
const [stateVariable, setStateVariable] = useState(initialValue);
```

Explanation

Part	Description
stateVariable	Stores the current value
setStateVariable	Updates the state value
useState()	React Hook used to create state
initialValue	Initial value of the state

Flow Diagram

Component Loads

|



Create State

|

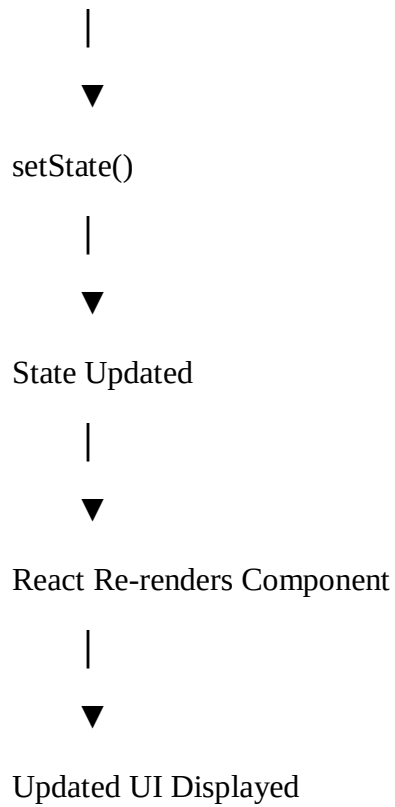


Display State

|



User Clicks Button



Example 3: Welcome Message

```
import { useEffect } from "react";
```

```
function App() {
```

```
  useEffect(() => {
```

```
    alert("Welcome to React!");
```

```
  }, []);
```

```
  return <h1>Hello Students</h1>;
```

```
}
```

```
export default App;
```

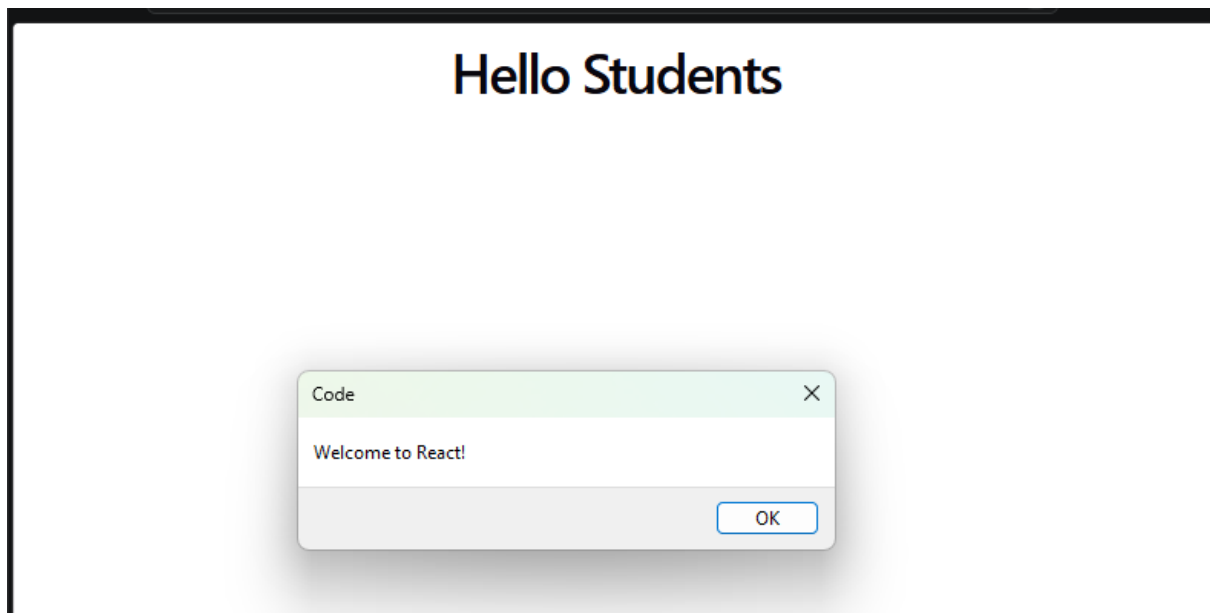
Output

When the page loads:

Welcome to React!

Then:

Hello Students



Example 4: Digital Clock using useEffect

```
import { useState, useEffect } from "react";
```

```
function App() {
```

```
  const [time, setTime] = useState(new Date());
```

```
useEffect(() => {

  const timer = setInterval(() => {

    setTime(new Date());

  }, 1000);

  return () => clearInterval(timer);

}, []);

return (

  <div>

    <h1>Digital Clock</h1>

    <h2>{time.toLocaleTimeString()}</h2>

  </div>

);
}

export default App;
```

Output

Digital Clock

10:45:20 AM

Updates every second.

Difference between useState and useEffect

useState()	useEffect()
Stores data	Performs side effects
Changes component state	Executes code after rendering
Causes re-render	Does not store data
Returns state and updater function	Takes a callback function

Real-Life Example

useState

A classroom attendance count:

Present Students = 45

↓

One student enters

↓

Present Students = 46

The count changes.

useEffect

When a classroom starts:

Class Starts

↓

Attendance Taken

↓

Projector Turned On

↓

Announcements Displayed

These actions happen after the class starts, similar to side effects after rendering.

Applications

useState

- Counter
- Calculator
- Login Form
- Shopping Cart
- Todo List

useEffect

- API Calls
- Digital Clock
- Weather App
- Online News Feed
- Document Title Updates

What is React Router?

Definition

React Router is a library used to navigate between different pages (components) in a React application **without refreshing the browser**.

It enables **Single Page Application (SPA)** navigation.

Why Do We Need React Router?

Without React Router:

- Only one page is displayed.
- Clicking links reloads the browser.

With React Router:

- Multiple pages can be displayed.
 - Navigation is faster.
 - The page does not reload.
-

Real-Life Example

Consider a college website.

- Home
- About
- Courses
- Contact

Clicking each menu should display a different page **without reloading**.

React Router makes this possible.

React Router Flow Diagram

Browser

|



React Router

|



URL Matches Route

|



Required Component Displayed

Features of React Router

- Client-side routing
- Fast navigation
- No page refresh

- Easy to create multiple pages
 - Supports dynamic routing
-

Important Components

Component	Purpose
BrowserRouter	Enables routing
Routes	Contains all routes
Route	Defines a route
Link	Creates navigation links
NavLink	Navigation with active styling
useNavigate	Navigates using JavaScript

Installation

Open Terminal

```
npm install react-router-dom
```

Wait until installation completes.

Project Structure

my-react-app

|

|— src

| └─ App.jsx

| └─ Home.jsx

| └─ About.jsx

| └─ Contact.jsx

| └─ main.jsx

Step 1: Create Home Component

Home.jsx

```
function Home() {  
  
  return (  
  
    <h2>Welcome to Home Page</h2>  
  
  );  
  
}  
  
export default Home;
```

Step 2: Create About Component

About.jsx

```
function About() {  
  
  return (  
  
    <h2>Welcome to About Page</h2>  
  
  );  
  
}  
  
export default About;
```

Step 3: Create Contact Component

Contact.jsx

```
function Contact() {  
  
  return (  
  
    <h2>Welcome to Contact Page</h2>  
  
  );  
  
}  
  
export default Contact;
```

contact.jsx to add content

```
function Contact() {  
  return (  
    <div>  
      <h2>Contact Page</h2>  
      <p>Email: cbit@gmail.com</p>  
    </div>  
  );  
}  
  
export default Contact;
```

Step 4: Modify App.jsx

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";  
  
import Home from "./Home";  
import About from "./About";  
import Contact from "./Contact";  
  
function App() {  
  return (  
    <BrowserRouter>  
  
      <h1>React Router Example</h1>
```

```
<nav>
```

```
<Link to="/">Home</Link>
```

```
{ " | " }
```

```
<Link to="/about">About</Link>
```

```
{ " | " }
```

```
<Link to="/contact">Contact</Link>
```

```
</nav>
```

```
<hr />
```

```
<Routes>
```

```
<Route path="/" element={<Home />} />
```

```
<Route path="/about" element={<About />} />
```

```
<Route path="/contact" element={<Contact />} />
```

```
</Routes>
```

```
</BrowserRouter>
```

```
);
```

```
}
```

```
export default App;
```

Step 5: main.jsx

Keep it as:

```
import React from "react";
```

```
import ReactDOM from "react-dom/client";
```

```
import App from "./App";
```

```
ReactDOM.createRoot(document.getElementById("root")).render(
```

```
<App />
```

```
);
```

Execution Steps

Step 1

Open your React project.

Step 2

Open Terminal.

```
C:\Users\CBIT\Desktop\reactrouter> npm.cmd create vite@latest
```

```
> npx
```

```
> create-vite
```

```
|
```

```
◇ Project name:
```

```
| RROUTER
```

```
|
```

```
◇ Package name:
```

```
| package.son
```

```
|
```

```
◇ Select a framework:
```

```
| React
```

```
|
```

```
◇ Select a variant:
```

```
| JavaScript
```

```
|
```

```
◇ Which linter to use?
```

```
| ESLint
```

```
|
```

```
◇ Install with npm and start now?
```

```
| Yes
```

```
|
```

```
◇ Scaffolding project in C:\Users\CBIT\Desktop\reactrouter\RROUTER...
```

|

◇ Installing dependencies with npm...

added 135 packages, and audited 136 packages in 1m

31 packages are looking for funding

run `npm fund` for details

found 0 vulnerabilities

|

◇ Starting dev server...

> package.son@0.0.0 dev

> vite

PS C:\Users\CBIT\Desktop\reactrouter\ cd RROUTER

Step 3

Install Router

npm install react-router-dom

Step 4

Run project

npm run dev

Step 5

Open

http://localhost:5173

Output

React Router Example

[Home](#) | [About](#) | [Contact](#)

Welcome to Home Page

Initially

React Router Example

Home | About | Contact

Welcome to Home Page

Click **About**

React Router Example

[Home](#) | [About](#) | [Contact](#)

Welcome to About Page

Welcome to About Page

Click **Contact**

Welcome to Contact Page

Notice that **the browser does not refresh.**

How React Router Works

User Clicks Link

|



React Router

|



Checks URL

|



Displays Matching Component

BrowserRouter

It enables routing in the application.

```
<BrowserRouter>
```

...

```
</BrowserRouter>
```

Routes

Contains all routing information.

```
<Routes>
```

...

```
</Routes>
```

Route

Maps a URL to a component.

```
<Route path="/about" element={<About />} />
```

Meaning:

If URL is

/about

display

About Component

Link

Used to navigate between pages.

```
<Link to="/about">
```

About

```
</Link>
```

Unlike an HTML `<a>` tag, Link changes the page **without reloading the browser**.

Applications

React Router is used in:

- College Website
 - Hospital Management System
 - Banking Application
 - Shopping Website
 - Employee Management
 - Student Portal
 - Online Examination System
-

Advantages

- Faster navigation
- Better user experience
- No browser refresh
- Easy to manage pages
- Supports dynamic routing

Introduction

A **form** is used to collect information from users such as:

- Student Registration
- Login
- Online Examination
- Banking Applications
- Shopping Websites
- Employee Registration

Before submitting the data, it is important to verify that the information entered is correct. This process is called **Form Validation**.

What is Form Validation?

Definition

Form Validation is the process of checking whether the user has entered valid and complete information before submitting the form.

Validation ensures that:

- Required fields are filled.
- Data is entered in the correct format.
- Incorrect information is rejected.
- Only valid data is submitted.

Why is Form Validation Required?

Form validation helps to:

- Prevent empty fields.
 - Improve data accuracy.
 - Enhance user experience.
 - Reduce server errors.
 - Improve application security.
-

Types of Validation

1. Client-Side Validation

Validation is performed in the browser before sending data to the server.

Examples:

- Required field
- Email format
- Password length

Advantages

- Fast response
- Immediate feedback
- Reduces server load

2. Server-Side Validation

Validation is performed on the server after the form is submitted.

Examples:

- Checking whether an email already exists.
- Verifying login credentials.
- Validating database records.

Controlled Components

In React, form elements are usually **controlled components**.

A controlled component is an input element whose value is controlled by the React state.

Example:

```
<input  
  type="text"  
  value={name}  
  onChange={handleChange}  
>
```

React stores the input value in **state**, making it easier to validate and manage the form.

Flow of Form Validation

User Enters Data

|



Input Stored in State

|



Validation Function Executes

|



Any Errors?

|

|

Yes

No

|

|

Show Errors Submit Form

React Hooks Used

useState()

The useState() hook stores:

- Form values
- Error messages

Example:

```
const [formData, setFormData] = useState({  
  name:"",  
  email:"",  
  password:""  
});
```

Event Handling

React uses events to capture user actions.

Event	Purpose
onChange	Detects input changes
onSubmit	Executes when the form is submitted
onClick	Detects button clicks

Validation Rules

A registration form commonly validates:

Name

- Cannot be empty
- Must contain alphabets

Email

- Cannot be empty
- Must follow email format

Example:

abc@gmail.com

Password

- Minimum 6 characters
- Can include letters, numbers, and symbols

Regular Expression for Email Validation

Example:

`^S+@S+\.S+\/`

This checks whether the email follows the pattern:

username@domain.extension

Example of valid emails:

student@gmail.com

abc@yahoo.com

test@cbit.ac.in

Displaying Error Messages

If validation fails, React displays an error message below the corresponding input field.

Example:

Name is required

Invalid Email

Password must be at least 6 characters

This helps users correct mistakes before submitting the form

Preventing Form Submission

Normally, clicking the **Submit** button refreshes the page.

React prevents this using:

```
event.preventDefault();
```

This keeps the user on the same page while validation is performed.

Advantages of Form Validation

- Prevents invalid input.
- Improves data quality.
- Reduces server-side errors.
- Enhances security.
- Provides a better user experience.
- Saves processing time.

Real-Time Applications

Form validation is used in:

- Student Registration System
- College Admission Portal

- Banking Applications
- Employee Registration
- Online Shopping
- Railway Reservation
- Hospital Management System
- Online Examination System

Difference between HTML Validation and React Validation

HTML Validation	React Validation
Uses HTML attributes like required	Uses JavaScript and React state
Limited customization	Highly customizable
Browser-controlled	Developer-controlled
Basic validation	Advanced validation with custom rules

Project Structure

rvalidation

```

|
|— src
|   |— App.jsx
|   |— main.jsx
|   |— index.css
|
|— index.html
|— package.json
  
```

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

App.jsx

```
import { useState } from "react";

function App() {

  const [formData, setFormData] = useState({

    name: "",

    email: "",

    password: ""

  });

  const [errors, setErrors] = useState({});
```

```
// Handle Input Changes
```

```
function handleChange(event) {
```

```
    const { name, value } = event.target;
```

```
    setFormData({
```

```
        ...formData,
```

```
        [name]: value
```

```
    });
```

```
}
```

```
// Form Validation
```

```
function validate() {
```

```
    let tempErrors = {};
```

```
    if (formData.name.trim() === "") {
```

```
        tempErrors.name = "Name is required";
```

```
    }
```

```
    if (formData.email.trim() === "") {
```

```
        tempErrors.email = "Email is required";
```

```
}
```

```
else if (!/^S+@\S+\.\S+/.test(formData.email)) {
```

```
    tempErrors.email = "Invalid Email";
```

```
}
```

```
if (formData.password.length < 6) {
```

```
    tempErrors.password = "Password must be at least 6 characters";
```

```
}
```

```
setErrors(tempErrors);
```

```
return Object.keys(tempErrors).length === 0;
```

```
}
```

```
// Submit Form
```

```
function handleSubmit(event) {
```

```
    event.preventDefault();
```

```
    if (validate()) {
```

```
alert("Form Submitted Successfully");
```

```
console.log(formData);
```

```
}
```

```
}
```

```
return (
```

```
<div style={{width:"400px",margin:"30px auto"}}>
```

```
<h2>Student Registration Form</h2>
```

```
<form onSubmit={handleSubmit}>
```

```
<label>Name</label><br/>
```

```
<input
```

```
type="text"
```

```
name="name"
```

```
value={formData.name}
```

```
onChange={handleChange}
```

```
/>
```

```
<br/>
```

```
<span style={{color:"red"}}>{errors.name}</span>
```

```
<br/><br/>
```

```
<label>Email</label><br/>
```

```
<input
```

```
type="text"
```

```
name="email"
```

```
value={formData.email}
```

```
onChange={handleChange}
```

```
/>
```

```
<br/>
```

```
<span style={{color:"red"}}>{errors.email}</span>
```

```
<br/><br/>
```

```
<label>Password</label><br/>
```

```
<input
```

```
  type="password"
```

```
  name="password"
```

```
  value={formData.password}
```

```
  onChange={handleChange}
```

```
/>
```

```
<br/>
```

```
<span style={{color:"red"}}>{errors.password}</span>
```

```
<br/><br/>
```

```
<button type="submit">
```

```
  Register
```

```
</button>
```

```
</form>
```

```
</div>
```

```
);
```

```
}
```

```
export default App;
```

Steps to Execute

Step 1

Open your React project in VS Code.

Step 2

Replace **App.jsx** with the Form Validation code.

Step 3

main.jsx .

Step 4

Open the terminal and run:

```
npm run dev
```

Step 5

Open the browser:

`http://localhost:5173`

If You Don't Have `index.css`

You have two options:

Option 1 (Recommended)

Create a file named **`index.css`** inside the **`src`** folder.

Example:

```
body {  
  font-family: Arial, sans-serif;  
  background-color: #f5f5f5;  
}
```

Option 2

If you don't want to use CSS, remove this line from `main.jsx`:

```
import './index.css';
```

Then your `main.jsx` becomes:

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import App from "./App";
```

```
ReactDOM.createRoot(document.getElementById("root")).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

Student Registration Form

Name

Name is required

Email

Email is required

Password

Password must be at least 6 characters

Register

Student Registration Form

Name

Name is required

Email

Email is required

Password

Password must be at least 6 characters

Register

Student Registration Form

Name

Name is required

Email

Email is required

Code

×

Form Submitted Successfully

OK

Student Registration Form

Name

Email

Invalid Email

Password